

SYSCRAFTERS 2026 · Design-First System Design Hackathon

SALESTORM

High-Scale E-Commerce Flash Sale — System Design
Blueprint

Business question: How can we handle thousands of simultaneous purchase requests for limited inventory without overselling, while keeping payment and order processing reliable?

Team: [Team Name]

System Architect: [Name 1] | LLD & Design Engineer: [Name 2]

Data & API Engineer: [Name 3] | Reliability Engineer: [Name 4]

Date: 5 October 2026

10,000 concurrent buyers · 100 units · 0 oversold · 0 double charges · 0 lost orders

Contents

1	Executive Summary
2	Requirements & Assumptions
3	High-Level Architecture
4	Containers, Components & Deployment
5	Concurrency & Inventory Reservation Design
6	Payment & Order Workflow Design
7	Low-Level Design: Class Model
8	SOLID Principles & Design Patterns
9	Database Design
10	API & Event Design
11	Scalability & Reliability
12	Security & Observability
13	Architecture Decision Records
14	Pitch Plan & Jury Q&A
15	Practical Scenario Walkthroughs
16	Appendix: Deliverables Index

1 Executive Summary

SALESTORM is the architecture for a limited-stock flash sale: **10,000 customers press "Buy Now" for 100 units of one product at the same moment**. The platform must survive the spike, sell at most 100 units, never create duplicate reservations, payments or orders, and drive every paid purchase to a valid order even when services fail. This document is the complete design blueprint: requirements, high-level and low-level design, database, APIs and events, concurrency control, payment and order workflows, scalability, reliability, security, observability, and the architecture decisions behind them.

The design in five ideas

Idea	How it works	What it guarantees
1. Filter at the edge	CDN + WAF serve cached pages and block bots; the API Gateway authenticates (JWT) and rate-limits 5 req/s per user; a virtual waiting room admits buyers at a controlled 20k/s with signed admission tokens.	The backend sees a smooth, bounded load instead of a 500k req/s spike.
2. One contention point	Only the Inventory & Reservation Service decides who gets stock. A Redis Lua script (atomic check + decrement + dedupe) rejects ~99% of requests in under 1 ms.	The database hot row is touched only ~100 times.
3. The database is the guarantee	<code>UPDATE inventory SET available = available - 1 ... WHERE available >= 1</code> in one ACID transaction with the reservation insert and an outbox event, backed by <code>CHECK (available_quantity >= 0)</code> .	Overselling is physically impossible.
4. Idempotency everywhere	Idempotency-Key header + UNIQUE constraints on reservations, payments and orders; the gateway receives a unique merchant reference; consumers dedupe on <code>event_id</code> .	No duplicate reservation, charge or order.
5. Recover asynchronously	After payment: transactional outbox → Kafka → idempotent consumers, retries, dead-letter queues and reconciliation jobs; compensation (release / refund) when a step cannot complete.	A paid order survives the Order Service being down for 30 s.

Expected outcome for the practical test case

Stock = 100 · concurrent users = 10,000 · payment success 95% · failures 5% · duplicates 2% · Order Service unavailable for 30 s

Metric	Result
Successful reservations at any time	≤ 100
Requests answered SOLD_OUT	~9,700, in under 50 ms
Duplicate requests (~200)	Replayed from the idempotency store — no new reservation
Failed payments (~5)	Units released and re-offered to the next buyers in the waiting room
Final units sold	Exactly 100 — never more

Double charges	0
Orders lost during the 30 s Order Service outage	0 — events wait in Kafka and are processed on recovery

How to read this document

Chapters 2–4 cover the problem and the high-level architecture. Chapters 5–6 contain the two most critical designs: concurrency-safe inventory reservation, and payment & order reliability. Chapters 7–10 cover the class model, SOLID and design patterns, the database, and the APIs and events. Chapters 11–12 cover scalability, reliability, security and observability. Chapter 13 records the architecture decisions and trade-offs, chapter 14 gives prepared answers to the jury questions, and chapter 15 walks through every practical scenario the jury will test.

2 Requirements & Assumptions

1. Problem Statement

SALESTORM runs limited-stock flash sales. At sale start, **10,000 customers press "Buy Now" for 100 units** of Product X within seconds. The platform must:

- survive the traffic spike without collapsing,
- sell **at most 100 units** (never oversell, never go negative),
- never create duplicate reservations, payments or orders,
- drive every paid purchase to a valid order, even when services fail.

Business question: *How do we handle thousands of simultaneous purchase requests for limited inventory without overselling, while keeping payment and order processing reliable?*

2. Scope – the Required Pipeline

Customer → Product Discovery → Cart → Inventory Check → Inventory Reservation → Checkout
→ Payment → Order → Fulfilment → Shipment → Notification → Delivery Tracking

In scope: everything in the pipeline above, plus sale/promotion rules, admission control (waiting room), observability, security. **Out of scope:** seller onboarding, returns/refunds UI, recommendation engine, warehouse robotics, tax engine internals (treated as an external call).

3. Functional Requirements

ID	Requirement	Pipeline stage
FR-01	Customers can browse/search products and view sale pages (price, stock badge, countdown).	Discovery
FR-02	Customers can add/remove items in a cart; flash-sale items are limited to 1 unit per customer .	Cart
FR-03	System shows an <i>approximate</i> live stock indicator (may lag by ≤ 1 s).	Inventory Check
FR-04	"Buy Now" creates a temporary reservation (hold) for the unit if stock remains; otherwise returns SOLD OUT immediately.	Reservation
FR-05	A reservation expires after 10 minutes if unpaid and the unit returns to available stock.	Reservation
FR-06	Repeated "Buy Now" from the same customer/request returns the same reservation (idempotent).	Reservation
FR-07	Checkout validates the reservation, applies sale price/coupon, creates the order in CREATED → PAYMENT_PENDING .	Checkout

ID	Requirement	Pipeline stage
FR-08	Customer pays via an external payment gateway (card/UPI/wallet); duplicate pay requests never double-charge.	Payment
FR-09	On payment success: reservation → CONFIRMED → SOLD , order → CONFIRMED . On failure/timeout: reservation → RELEASED , order → CANCELLED .	Payment / Order
FR-10	Confirmed orders flow to fulfilment → shipment (PROCESSING → SHIPPED → OUT_FOR_DELIVERY → DELIVERED).	Fulfilment / Shipment
FR-11	Customers receive notifications (email/SMS/push) on reservation, payment, order and shipment events.	Notification
FR-12	Customers can track order and delivery status.	Tracking
FR-13	Admins can create a sale (product, stock, price, start/end time, per-customer limit).	Sale mgmt
FR-14	Every state change of inventory, reservation, payment and order is audited .	Cross-cutting

4. Non-Functional Requirements (measurable)

Category	Target	Type
Inventory correctness	<code>sold_quantity ≤ initial_stock</code> and <code>available_quantity ≥ 0</code> at all times — 0 oversells	Strict guarantee
Idempotency	0 duplicate reservations / payments / orders for the same idempotency key	Strict guarantee
Order completeness	Every successful payment reaches a valid order state (CONFIRMED or refunded) within 5 min	Strict guarantee (eventual)
Throughput – normal	~10,000 req/s	Target
Throughput – flash sale	Absorb up to 500,000 req/s at the edge; ≥ 20,000 reservation attempts/s at Inventory	Target
Latency – reserve API	p99 < 200 ms (sold-out response p99 < 50 ms)	Target
Latency – browse (cached)	p95 < 100 ms	Target
Latency – payment	p99 < 3 s end-to-end (gateway-bound)	Target
Availability	Checkout/Reserve/Payment 99.95% ; Browse 99.99% (CDN)	Target
Consistency	Strong for inventory, payment, order rows; eventual (≤ 1 s) for stock display, search, notifications	Design rule
Durability / Recovery	RPO = 0 for payments & orders (sync replica); RTO < 5 min for DB failover	Target
Reservation release	Expired holds released within ≤ 30 s of <code>expires_at</code>	Target

Category	Target	Type
Security	TLS 1.2+, OAuth2/JWT auth, PCI-DSS scope minimised (tokenised card data – we never store PAN)	Strict
Observability	100% of checkout/payment/order requests traced; alert within 1 min on inventory mismatch	Target

5. Assumptions

1. **One sale = one SKU** (Product X) with **100 units**; one warehouse. Design generalises to many SKUs by sharding on `sku_id`.
2. **Per-customer limit = 1 unit** for the flash sale (common practice; also prevents hoarding bots).
3. Customers must be **logged in** before "Buy Now" (no guest checkout during flash sales).
4. Traffic profile: ~70% of requests arrive in the **first 10 seconds** of the sale; most of them are page refreshes and stock polling.
5. Payment success rate $\approx$ **95%**, failure $\approx$ **5%**, duplicate client requests $\approx$ **2%** (from the practical test case).
6. External payment gateway supports **idempotency / merchant reference** and a **status query API** (true for Razorpay, Stripe, PayU, etc.).
7. Payment gateway latency 300 ms–3 s; it may time out or be unavailable.
8. Order Service may be **unavailable for up to 30 s** (practical test case) – design must tolerate this with no lost orders.
9. Cloud deployment (e.g. AWS / GCP) with managed Kubernetes, managed PostgreSQL, Redis and Kafka.
10. Reservation hold time = **10 minutes** (business-configurable per sale).

6. Constraints

- Inventory is a **single hot record** per SKU $\rightarrow$ classic hotspot; cannot be solved by adding app servers alone.
- Payment gateway is **external** – we cannot make it transactional with our DB $\rightarrow$ requires idempotency + reconciliation.
- Distributed services $\rightarrow$ **no distributed (2PC) transactions**; consistency via local transactions + events (Saga / Outbox).
- Hackathon scope: design-first; implementation optional.

7. Expected Traffic (back-of-envelope)

Quantity	Estimate
Concurrent buyers at T0	10,000 (test case) $\rightarrow$ design headroom for 50 $\times$ = 500,000
Page/stock polls	500k users $\times$ 1 req/s = 500k req/s $\rightarrow$ served by CDN + Redis cache
"Buy Now" attempts	burst of 10k–500k in ~5 s $\rightarrow$ 2k–100k req/s $\rightarrow$ waiting room admits at a controlled rate (e.g. 20k/s)
Successful reservations	100 (hard cap)
Payment calls	~100 + retries ($\approx$ 5 failures may re-release ~5 units to others)

Quantity	Estimate
Orders written	~100 → tiny write load; the contention , not volume, is the problem

Key insight: The flash sale is a *read-heavy, contention-heavy, write-light* problem. 99% of buyers must be told "sold out" **fast and cheaply**, while the 100 winners must be processed **correctly**.

8. Critical Dependencies

Dependency	Why critical	Mitigation
Redis (stock gate)	Absorbs contention	Redis Cluster + replica; DB remains source of truth
PostgreSQL (inventory, orders, payments)	Source of truth	Sync replica, auto-failover
Kafka (events)	Order/notification flow	3 brokers, RF = 3, <code>acks=all</code>
Payment gateway	Money movement	Circuit breaker, timeouts, reconciliation, optional 2nd provider
Auth / Identity provider	Login before buy	Token validation at gateway (stateless JWT)

9. Strict Guarantees vs Targets (summary)

Strict guarantees (must never be violated)	Targets (best effort, measured)
No overselling (≤ 100 sold)	Latency p99 numbers
Inventory never negative	500k req/s throughput
No duplicate reservation / payment / order per idempotency key	99.95% availability
Every captured payment → order or refund	Release of expired holds within 30 s
No card data stored (PCI)	Notification delivery within 10 s

3 High-Level Architecture

This chapter presents the system from the outside in: the system context (who and what SALESTORM talks to), then the high-level architecture of the "Buy Now" request path, followed by the reasoning behind every component.

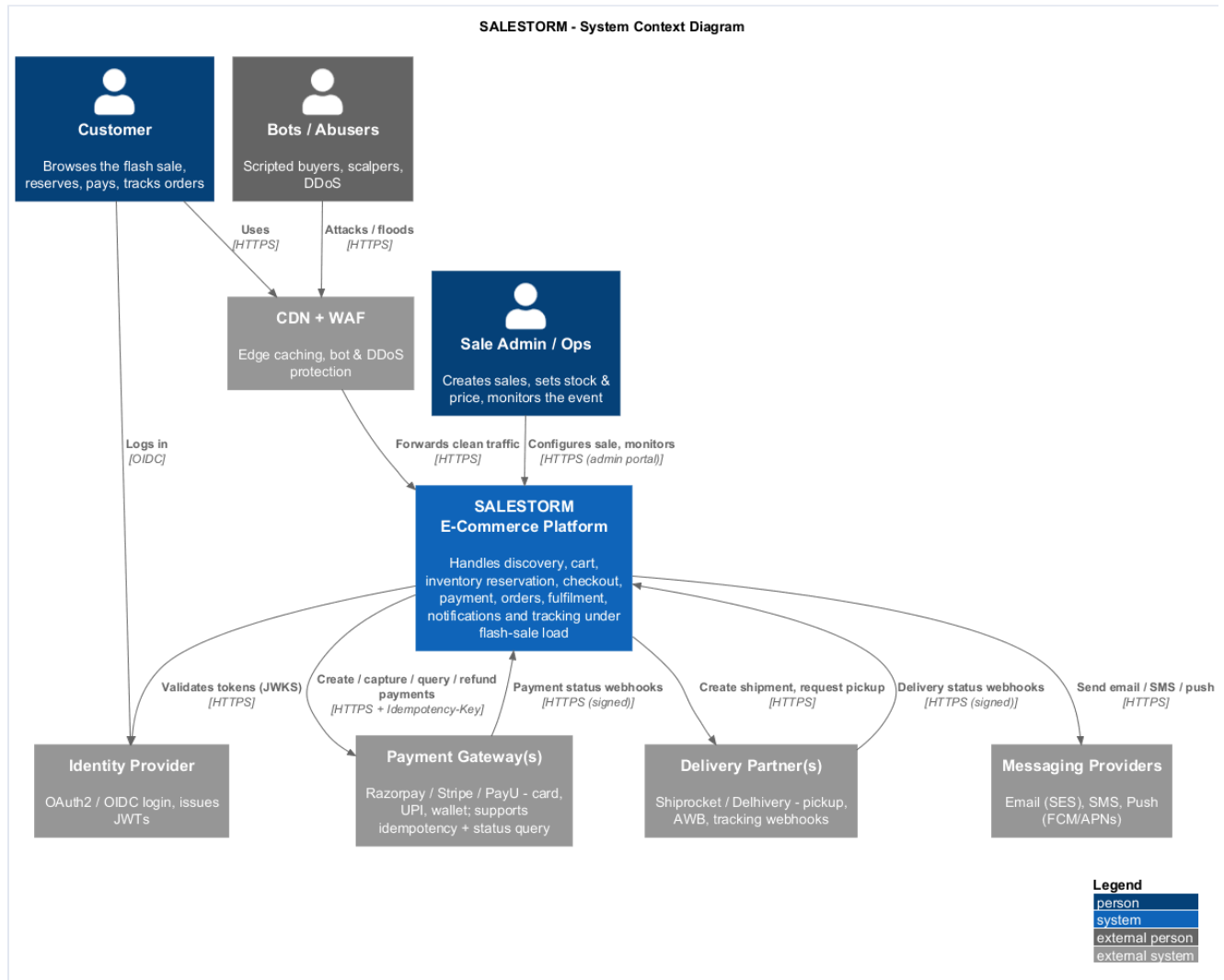


Figure 3.1 – System Context Diagram

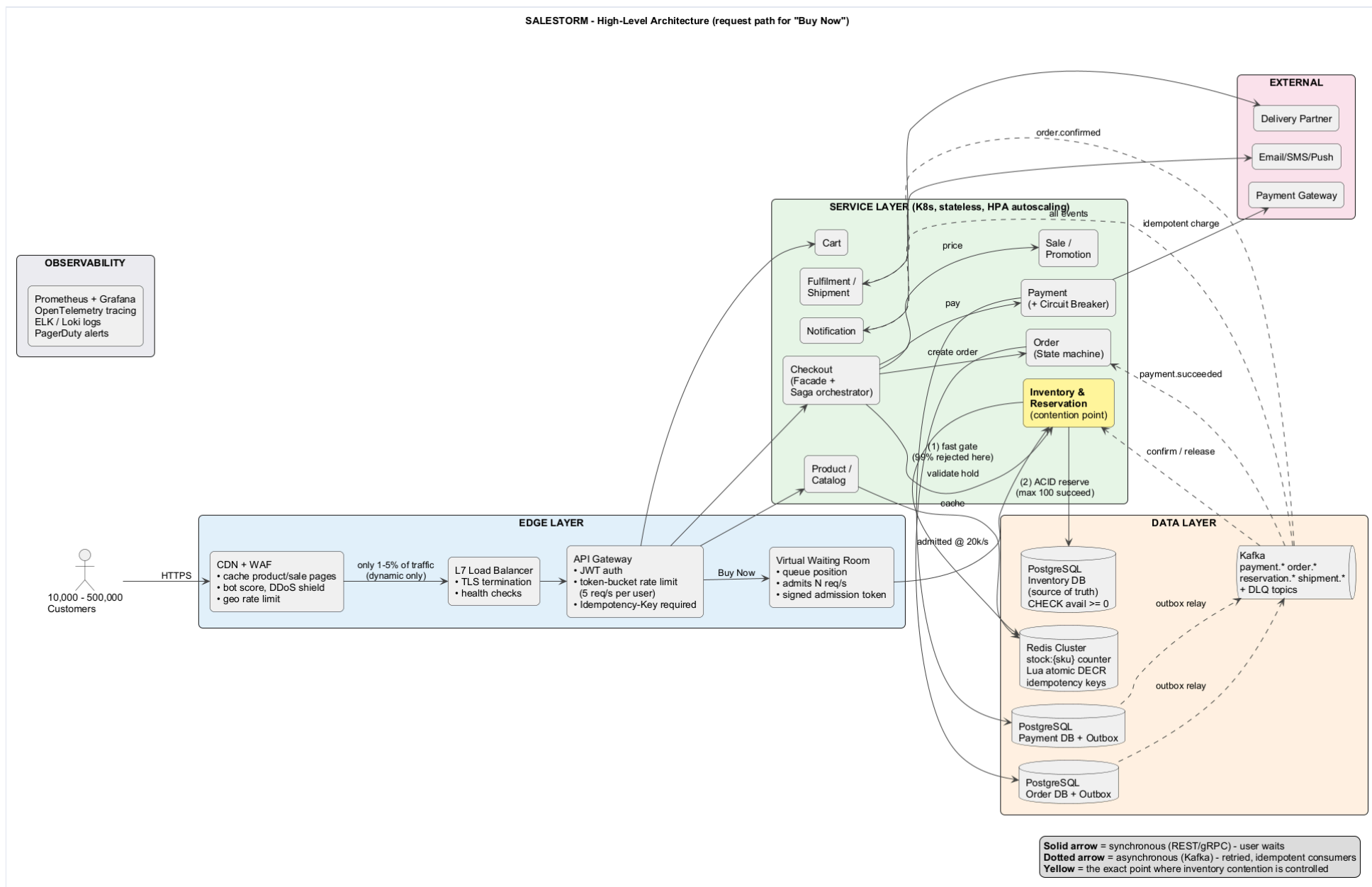
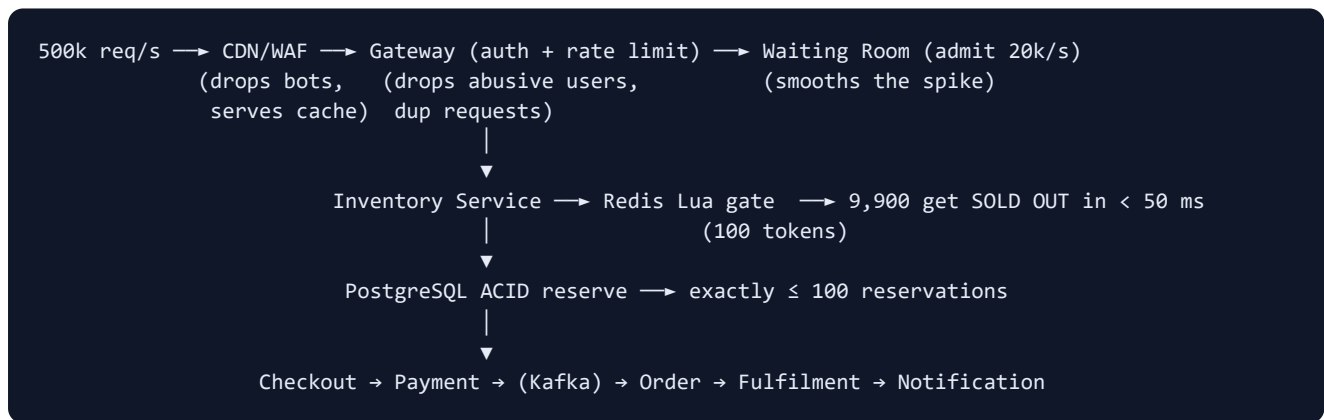


Figure 3.2 – High-Level Architecture (request path for "Buy Now"). Solid arrows are synchronous, dotted arrows are asynchronous Kafka events; the yellow box is the single point where inventory contention is controlled.

1. The Core Idea in One Sentence

Filter the flood at the edge, serialise contention at exactly one place (Inventory), and make everything after the reservation asynchronous, idempotent and recoverable.



2. Why Each Major Component Exists

Component	Why it exists	What breaks without it
CDN + WAF	Serves product/sale pages & static assets from edge; blocks bots/DDoS. ~95% of requests never reach us.	Origin overwhelmed by page refreshes.
Load Balancer	TLS termination, spreads load across gateway pods, health-based failover across AZs.	Single entry pod = SPOF.
API Gateway	Central JWT auth, per-user/IP rate limiting, routing, X-Request-ID & Idempotency-Key enforcement.	Every service re-implements auth; no abuse protection.
Waiting Room	Converts an uncontrolled spike into a steady admitted rate (e.g. 20k/s) with fair FIFO-ish ordering. Admission token is signed (HMAC) and short-lived.	Inventory & DB see the full spike; latency explodes, timeouts → retries → retry storm.
Product / Catalog	Read-heavy browsing, search. Cache-aside on Redis + CDN.	—
Sale / Promotion	Owns sale window, sale price, coupons, per-customer limit (Strategy pattern for pricing).	Pricing logic leaks into checkout.
Cart	Normal shopping flow. Flash-sale "Buy Now" bypasses the cart to shorten the critical path.	—
Inventory & Reservation	The single place where contention is controlled. Atomic reserve, confirm, release; expiry sweeper.	Overselling.
Checkout (Facade/Saga orchestrator)	One entry point that coordinates reservation → pricing → order → payment.	Client must call 4 services; partial failures uncoordinated.
Payment	Isolates money movement: idempotency, gateway adapters, circuit breaker, reconciliation.	Duplicate charges, gateway outages cascade.
Order	Owns order state machine; consumes payment events.	No single source of order truth.

Component	Why it exists	What breaks without it
Fulfilment / Shipment	Courier booking, tracking webhooks.	—
Notification	Async fan-out to email/SMS/push with retries.	Slow providers block checkout.
Redis Cluster	In-memory atomic counter (Lua) absorbs ~10k-500k contention attempts at μ s latency; also rate-limit buckets & idempotency cache.	All contention hits the DB row lock.
PostgreSQL (per service)	ACID source of truth ; <code>CHECK (available_quantity >= 0)</code> makes overselling <i>physically impossible</i> .	No hard guarantee.
Kafka	Durable, replayable events decouple payment → order → fulfilment → notification. Survives the Order Service being down for 30 s.	Synchronous chain: one failure loses the order.
Outbox tables	Atomically "save state + publish event" (no dual-write problem).	Payment saved but event lost (or vice versa).
Observability stack	Metrics, traces, logs, alerts for inventory mismatch / payment failures / queue lag.	Flying blind during the sale.

3. Synchronous vs Asynchronous

Interaction	Mode	Why
Browse / search	Sync (cached)	User waits; read-only.
Reserve (Buy Now)	Sync	Customer needs an immediate yes/no; consistency must be decided <i>now</i> .
Checkout → Inventory (validate hold)	Sync	Must not charge for an expired hold.
Checkout → Order (create <code>PAYMENT_PENDING</code>)	Sync	Order ID needed as payment reference.
Checkout → Payment → Gateway	Sync with timeout (≤ 10 s)	User is on the payment page; timeout falls back to async reconciliation.
Gateway → Payment (webhook)	Async	Final truth of payment status.
Payment → Order / Inventory (confirm, release)	Async (Kafka)	Must survive Order Service downtime; retried; idempotent.
Order → Fulfilment → Shipment	Async	Long-running, external partners.
Anything → Notification	Async	Never block a business transaction on email/SMS.

Rule of thumb we used: *Sync where the user needs the answer to make a decision; async where the system needs the work done reliably.*

4. Where Requests Are Throttled, Queued or Rejected

Layer	Mechanism	Rejection response
CDN/WAF	Bot score, IP reputation, geo/IP rate limits	403 / challenge
API Gateway	Token bucket: 5 req/s per user, 50 req/s per IP; missing JWT	429 / 401
Waiting Room	Queue; admit fixed rate; queue closed once stock gate shows 0	202 "in queue" → later "SOLD OUT"
Inventory (Redis gate)	<code>stock:{sku}</code> reaches 0 → instant reject	409 <code>SOLD_OUT</code>
Inventory (DB)	Conditional update affects 0 rows	409 <code>SOLD_OUT</code>
Idempotency	Same key → return original result	200 (replayed)

5. Bottlenecks and How We Address Them

Likely bottleneck	Why	Mitigation
Hot inventory row / key	All 10k requests target one SKU	Redis Lua gate rejects 99% in memory; only ≤ 100 + small retry margin reach the DB row; optional stock bucketing (split 100 units into 10 keys $\times$ 10) for 50 $\times$ load
DB connections	Thundering herd of pods	PgBouncer pooling; waiting-room admission caps concurrency
Payment gateway	External, rate-limited, slow	Only ~ 100 payments; circuit breaker + timeout + reconciliation; second provider via Strategy/Adapter
Kafka consumer lag	Burst of events	Partition by <code>order_id</code> ; scale consumers to partition count
Notification provider	Slow / rate-limited	Async queue, retries with backoff, DLQ

6. Horizontal Scaling & State

- All services are **stateless** → scale on Kubernetes HPA (CPU/RPS). Pre-scale 30 minutes before the sale (scheduled scaling) — autoscaling is too slow for a 5-second spike.
- State lives only in:** Redis (ephemeral, rebuildable), PostgreSQL (truth), Kafka (durable log).
- Sessions = stateless JWT; no sticky sessions needed.
- Sharding key for multi-SKU sales: `sku_id` (Redis hash slot + DB partition).

7. Trade-offs We Accept

We chose	We sacrifice	Why acceptable
Strong consistency for inventory	Some availability: if the Inventory DB primary is down, reservations pause (≈ 30 s failover)	Overselling is worse than a brief pause
Waiting room	Users wait / see queue position	Fairness + system stability beats a crash

We chose	We sacrifice	Why acceptable
Async order confirmation	Order shows <code>PAYMENT_PENDING</code> for a few seconds	Survives Order Service failure with zero lost orders
Redis + DB double gate	Two systems to keep in sync; reconciliation job needed	Redis gives speed, DB gives the guarantee
Microservices	Operational complexity, distributed tracing needed	Independent scaling of the hot path (Inventory) from cold paths

4 Containers, Components & Deployment

The container diagram shows every deployable service, data store and message channel, with synchronous and asynchronous interactions labelled. The component diagram opens up the four critical services. The deployment diagram shows where everything runs.

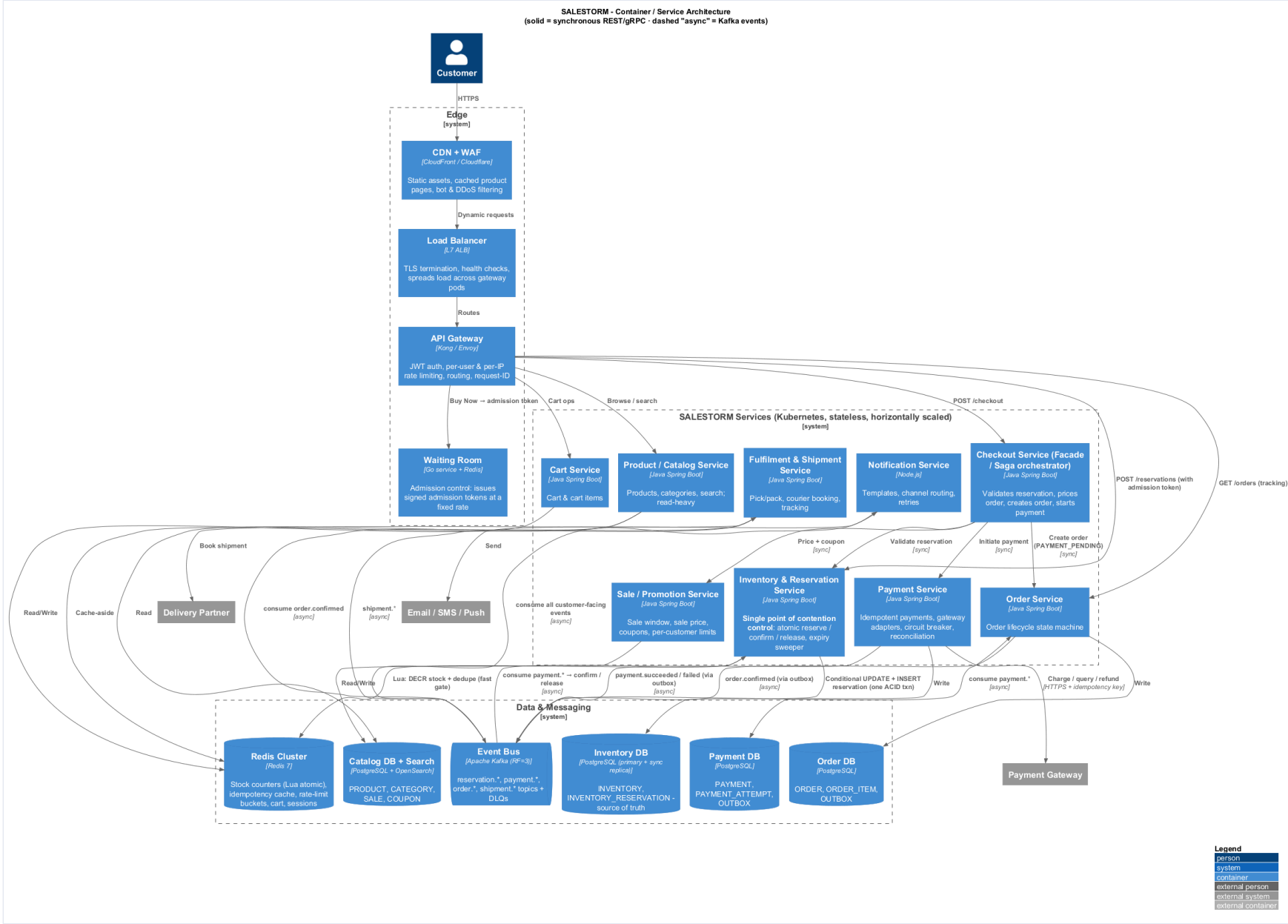


Figure 4.1 – Container / Service Architecture

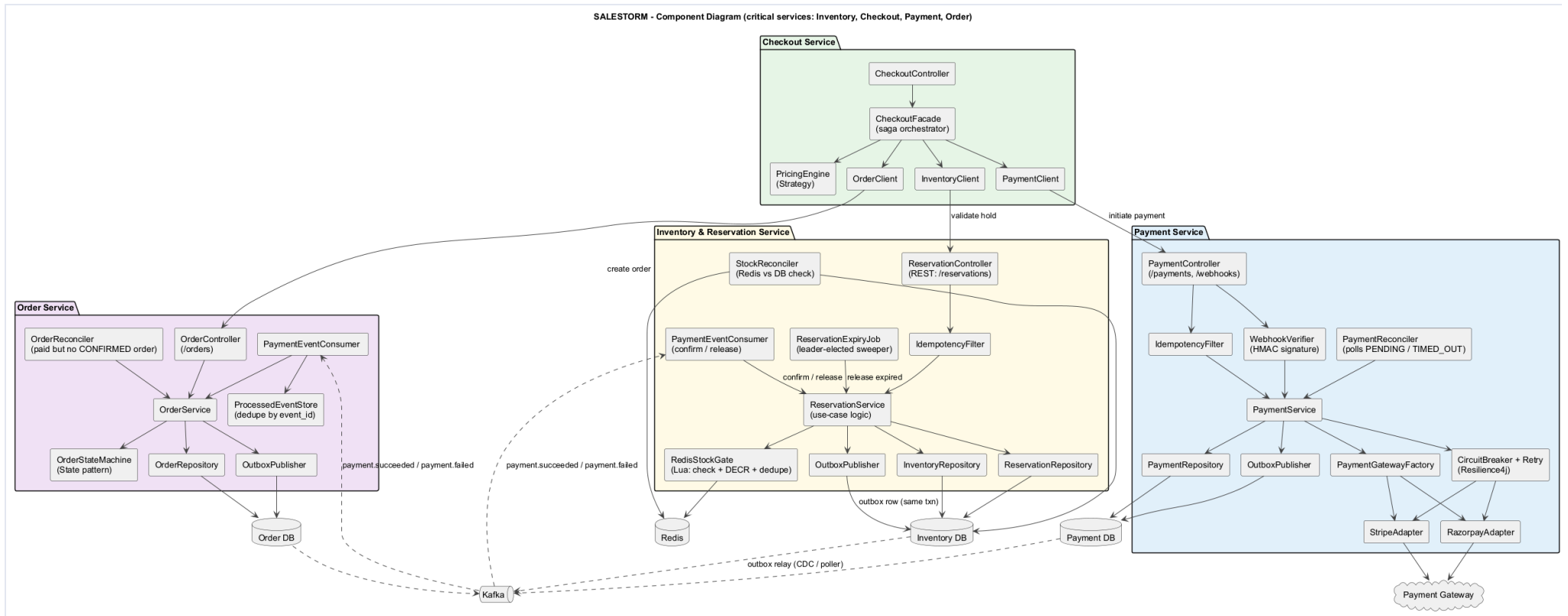


Figure 4.2 – Component Diagram of Inventory, Checkout, Payment and Order services

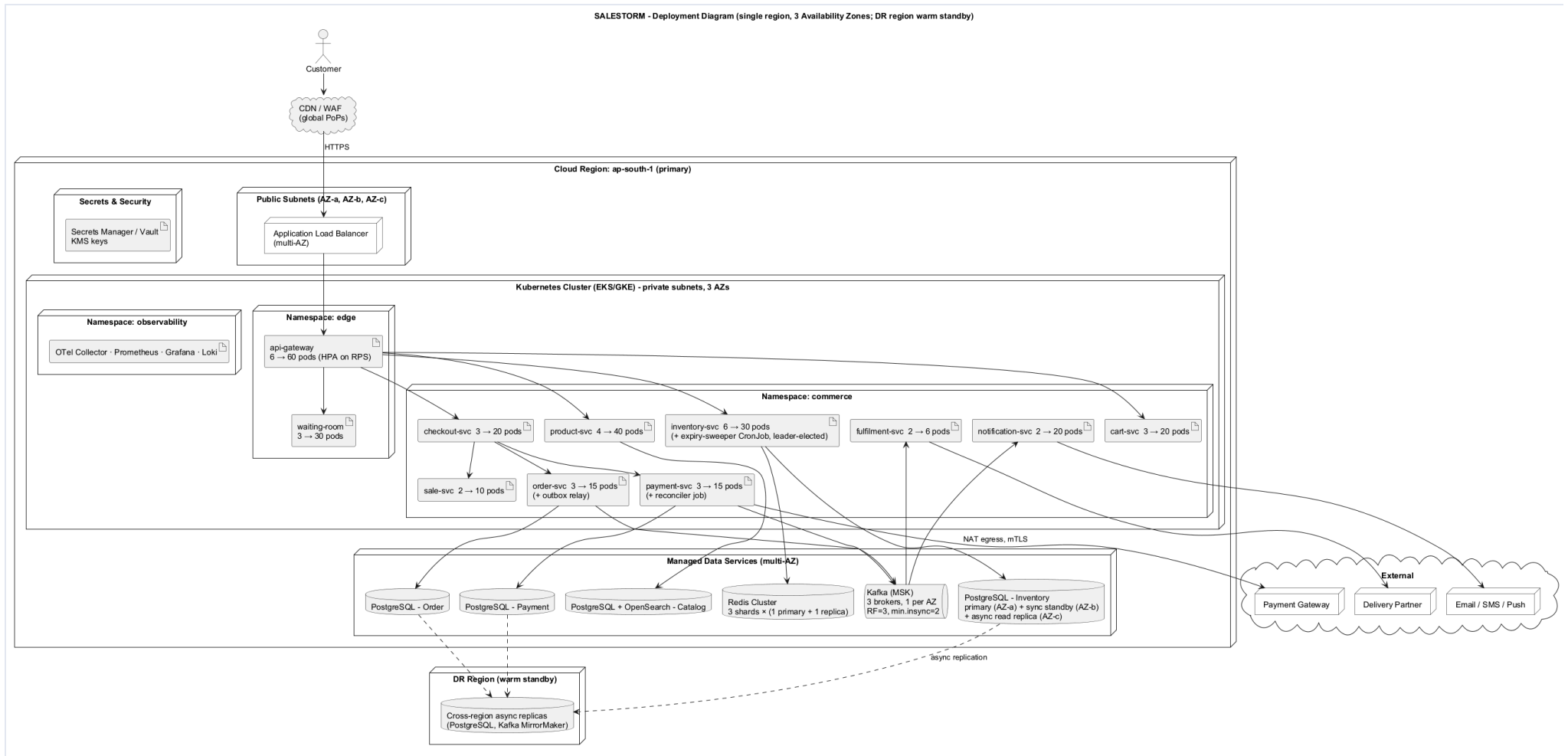


Figure 4.3 – Deployment Diagram (single region, 3 availability zones, warm DR region)

Service boundaries and responsibilities

Service	Owns (data)	Responsibility	Scales with
Product / Catalog	PRODUCT, CATEGORY	Browse, search, product detail	Read traffic (cached)
Sale / Promotion	SALE, COUPON	Sale window, sale price, coupons, per-customer limits	Low
Cart	CART, CART_ITEM	Cart operations (flash-sale Buy Now bypasses the cart)	Normal traffic
Inventory & Reservation	INVENTORY, INVENTORY_RESERVATION	Atomic reserve / confirm / release, expiry sweeper, stock reconciler	Contention (hot path)
Checkout	— (stateless)	Facade and saga orchestrator for reserve → price → order → pay	Checkouts (~100s)
Payment	PAYMENT, OUTBOX	Idempotent payments, gateway adapters, circuit breaker, reconciliation, refunds	Payments (~100s)
Order	ORDER, ORDER_ITEM, PROCESSED_EVENT, OUTBOX	Order state machine, consumes payment events	Orders
Fulfilment / Shipment	SHIPMENT	Courier booking, tracking webhooks	Orders
Notification	NOTIFICATION	Email / SMS / push with retries	Events

5 Concurrency & Inventory Reservation Design

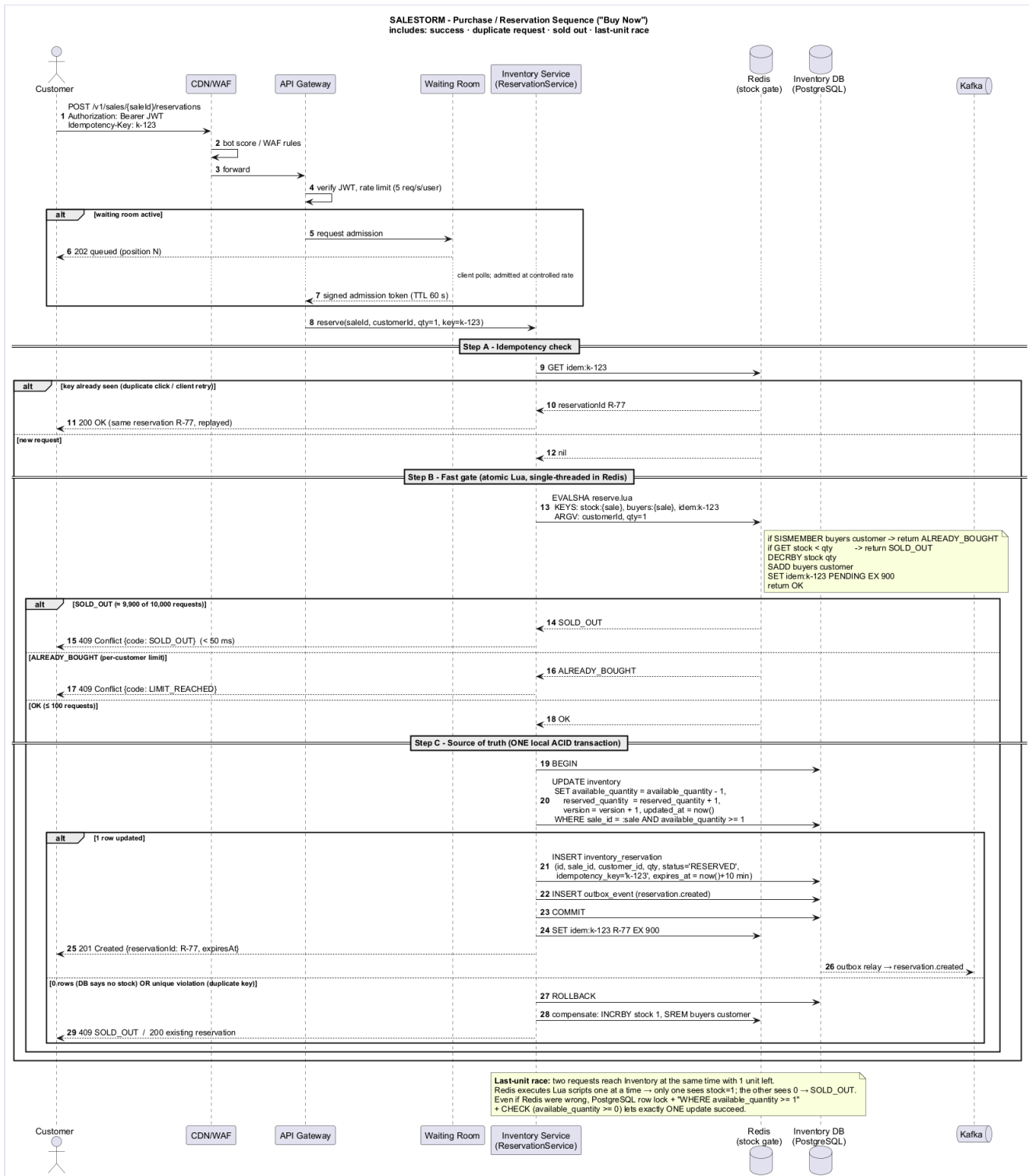


Figure 5.1 – Purchase / Reservation Sequence (success, duplicate, sold out, last-unit race)

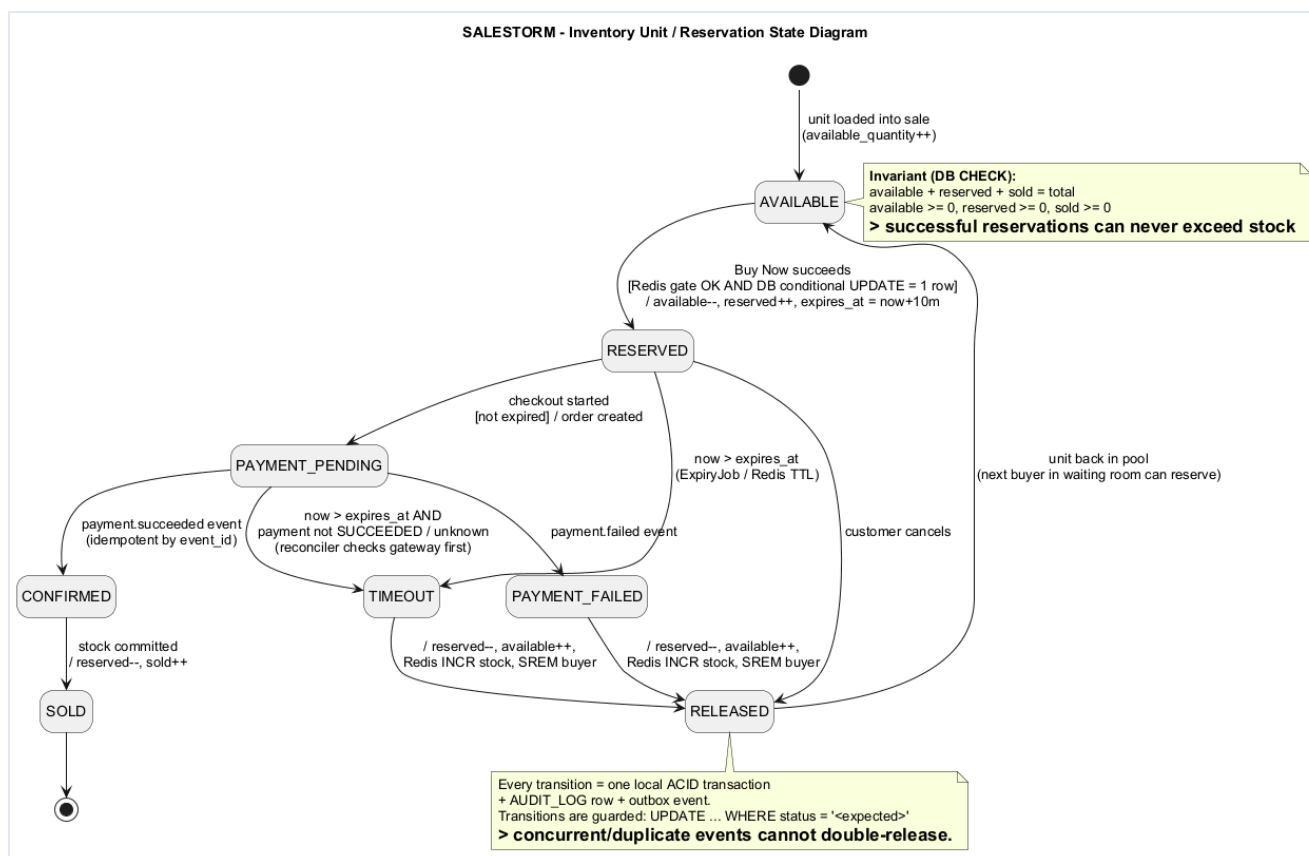


Figure 5.2 – Inventory Unit / Reservation State Diagram

1. The Guarantee

Successful reservations + sales can never exceed 100, and `available_quantity` can never be negative — regardless of how many requests arrive or in what order.

Enforced by **two layers**:

Layer	Role	Guarantee level
Layer 1 – Redis Lua gate	<i>Performance.</i> Absorbs 10k–500k attempts in memory; rejects ~99% in < 1 ms.	Fast, <i>advisory</i>
Layer 2 – PostgreSQL conditional UPDATE + CHECK constraints	<i>Correctness.</i> Single row, single ACID transaction, row-level lock.	Hard guarantee

Answer to "Where exactly is inventory consistency guaranteed?" In the Inventory DB, at the statement `UPDATE inventory ... WHERE available_quantity >= :qty` inside the reservation transaction, backed by `CHECK (available_quantity >= 0)`. Redis only decides *who is allowed to try*.

2. Comparing Concurrency-Control Approaches

#	Approach	How it works	Pros	Cons under 10k → 100	Verdict
A	Pessimistic lock <code>SELECT ... FOR UPDATE</code> then <code>UPDATE</code>	Lock row, check, decrement	Simple, strictly correct	All 10k transactions queue on one row lock ; each holds lock for a round-trip → ~2–5k TPS max, connection pool exhaustion, timeouts	✗ alone
B	Optimistic lock (version column)	Read version, <code>UPDATE ... WHERE version = :v</code>	No locks held while thinking	Under heavy contention almost every attempt fails and retries → retry storm, wasted DB work; livelock risk	✗ for hot SKU
C	Atomic conditional <code>UPDATE SET avail = avail-1 WHERE avail >= 1</code>	DB checks + decrements in one statement; row lock held only microseconds	Correct, no read-modify-write gap, no retry loop	Still serialises on one row → DB CPU/locks become bottleneck at 10k+ concurrent	✓ as source of truth
D	Redis atomic counter (Lua script)	Single-threaded Redis runs check + DECR + dedupe atomically	~100k+ ops/s on one key, sub-ms	In-memory; must be kept in sync with DB; can be lost on failover	✓ as fast gate
E	Queue-based serialisation (Kafka partition per SKU, single consumer)	All buy requests become messages; one consumer processes in order	Perfect ordering, no locks	Async result → user waits/polls; extra latency; consumer is SPOF per SKU	⚠ alternative for 50×+
F	Distributed lock (Redlock)	Lock around the decrement	—	Slow, complex, still single-file; correctness debated	✗

Selected: D + C (Redis Lua fast gate in front of an atomic conditional UPDATE). *Why:* D removes 99% of contention cheaply; C guarantees correctness with only ~100–150 transactions ever touching the hot row. Option E is our documented scale-out path (see ADR-003).

3. Data Model (concurrency-critical fields)

```
CREATE TABLE inventory (  
  inventory_id      UUID PRIMARY KEY,  
  product_id       UUID NOT NULL REFERENCES product(product_id),  
  sale_id          UUID NOT NULL REFERENCES sale(sale_id),  
  total_quantity   INT  NOT NULL CHECK (total_quantity > 0),  
  available_quantity INT  NOT NULL CHECK (available_quantity >= 0),  
  reserved_quantity INT  NOT NULL DEFAULT 0 CHECK (reserved_quantity >= 0),  
  sold_quantity    INT  NOT NULL DEFAULT 0 CHECK (sold_quantity >= 0),  
  version          BIGINT NOT NULL DEFAULT 0,  
  updated_at       TIMESTAMPTZ NOT NULL DEFAULT now(),  
  CONSTRAINT uq_inventory_sale UNIQUE (product_id, sale_id),  
  CONSTRAINT chk_conservation CHECK (available_quantity + reserved_quantity + sold_quantity =  
total_quantity)  
);  
  
CREATE TABLE inventory_reservation (  
  reservation_id   UUID PRIMARY KEY,  
  inventory_id     UUID NOT NULL REFERENCES inventory(inventory_id),  
  sale_id         UUID NOT NULL,  
  customer_id     UUID NOT NULL REFERENCES customer(customer_id),  
  quantity        INT  NOT NULL CHECK (quantity > 0),  
  status          VARCHAR(20) NOT NULL,  
  idempotency_key  VARCHAR(64) NOT NULL UNIQUE,          -- duplicate request protection  
  expires_at      TIMESTAMPTZ NOT NULL,  
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),  
  updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
-- one active reservation per customer per sale (per-customer limit = 1)  
CREATE UNIQUE INDEX uq_active_res_per_customer ON inventory_reservation (sale_id, customer_id)  
  WHERE status IN ('RESERVED', 'PAYMENT_PENDING', 'CONFIRMED', 'SOLD');  
-- expiry sweeper  
CREATE INDEX ix_res_expiry ON inventory_reservation (status, expires_at)  
  WHERE status IN ('RESERVED', 'PAYMENT_PENDING');
```

chk_conservation is our strongest invariant: units cannot appear or vanish.

4. Layer 1 – Redis Lua Gate

Keys (hash-tagged so they live on the same Redis Cluster slot): - **stock:{sale42}** – integer, initialised to 100 at sale start - **buyers:{sale42}** – set of customer IDs holding/bought a unit - **idem:{sale42}:<key>** – idempotency record, TTL 15 min

```
-- reserve.lua KEYS[1]=stock KEYS[2]=buyers KEYS[3]=idem  
-- ARGV[1]=customerId ARGV[2]=qty ARGV[3]=ttlSeconds  
local existing = redis.call('GET', KEYS[3])  
if existing then return {'DUPLICATE', existing} end  
if redis.call('SISMEMBER', KEYS[2], ARGV[1]) == 1 then return {'LIMIT_REACHED'} end  
local stock = tonumber(redis.call('GET', KEYS[1]) or '0')  
if stock < tonumber(ARGV[2]) then return {'SOLD_OUT'} end  
redis.call('DECRBY', KEYS[1], ARGV[2])  
redis.call('SADD', KEYS[2], ARGV[1])  
redis.call('SET', KEYS[3], 'PENDING', 'EX', ARGV[3])  
return {'OK'}
```

Redis executes a Lua script **atomically and single-threaded** → no two scripts interleave → the 101st caller always sees `stock = 0`.

5. Layer 2 – The ACID Reservation Transaction (transaction boundary)

```
BEGIN; -- READ COMMITTED is sufficient: the UPDATE re-checks the predicate on the locked row
UPDATE inventory
  SET available_quantity = available_quantity - :qty,
      reserved_quantity = reserved_quantity + :qty,
      version = version + 1, updated_at = now()
  WHERE sale_id = :sale AND available_quantity >= :qty;
-- application: if rowcount = 0 → ROLLBACK, return SOLD_OUT

INSERT INTO inventory_reservation (reservation_id, inventory_id, sale_id, customer_id, quantity,
                                   status, idempotency_key, expires_at)
VALUES (:rid, :inv, :sale, :cust, :qty, 'RESERVED', :idemKey, now() + interval '10 minutes');
-- unique violation on idempotency_key or (sale_id, customer_id) → ROLLBACK, return existing

INSERT INTO outbox_event (event_id, aggregate_type, aggregate_id, event_type, payload)
VALUES (:eid, 'Reservation', :rid, 'reservation.created', :json);

INSERT INTO audit_log (...) VALUES (...);
COMMIT;
```

Transaction boundary = one Inventory DB, one transaction. Stock decrement, reservation row, outbox event and audit row commit or roll back together. No distributed transaction is ever needed for reservation.

If DB fails after Redis said OK: compensate Redis (`INCRBY stock` , `SREM buyers` , `DEL idem`). If the compensation itself is lost (pod crash), Redis shows *fewer* units than DB → the system **undersells temporarily** (safe direction), and the `StockReconciler` (every 10 s) resets `stock:{sale} = DB available_quantity`.

Design principle: every failure mode errs toward **underselling** (recoverable) never **overselling** (unrecoverable).

6. Reservation Lifecycle Operations

Operation	Trigger	DB statement (guarded)	Redis
Create	<code>POST /reservations</code>	see §5	Lua gate
Mark payment pending	Checkout	<code>UPDATE reservation SET status='PAYMENT_PENDING' WHERE id=? AND status='RESERVED' AND expires_at > now()</code>	–
Confirm → Sold	<code>payment.succeeded</code> event	<code>UPDATE reservation SET status='SOLD' WHERE id=? AND status IN ('RESERVED','PAYMENT_PENDING') + UPDATE inventory SET reserved=reserved-1, sold=sold+1 (same txn)</code>	–
Release	<code>payment.failed</code> , expiry, cancel	<code>UPDATE reservation SET status='RELEASED' WHERE id=? AND status IN ('RESERVED','PAYMENT_PENDING') → if 1 row: UPDATE inventory SET reserved=reserved-1, available=available+1</code>	<code>INCRBY stock</code> , <code>SREM buyers</code>

Operation	Trigger	DB statement (guarded)	Redis
Expire	ReservationExpiryJob every 5 s	SELECT ... WHERE status IN (...) AND expires_at < now() FOR UPDATE SKIP LOCKED LIMIT 100 → release each	same

The `WHERE status = <expected>` guard makes every transition **idempotent and race-safe**: if a duplicate `payment.failed` event or the expiry job and a late payment race, only one UPDATE matches; the loser affects 0 rows and does nothing.

Expiry vs late payment race: the expiry job does **not** release a `PAYMENT_PENDING` reservation whose payment is `PENDING/TIMED_OUT` ; it asks Payment's reconciler first. If payment later succeeds for an already-released unit, the order is **refunded** (compensation) — see Payment & Order design.

7. Handling Duplicate Requests

Duplicate type	Detection	Result
Same click retried (same Idempotency-Key)	Redis <code>idem:</code> key → DB <code>UNIQUE(idempotency_key)</code>	Returns the same reservation (200, replayed)
Same customer, new key (two tabs)	Redis <code>buyers</code> set → DB partial unique (<code>sale_id</code> , <code>customer_id</code>)	409 <code>LIMIT_REACHED</code>
Two concurrent identical requests	Lua atomicity; DB unique constraint is the final arbiter	Exactly one reservation

8. Trace: 10,000 Requests for 100 Units

Step	Count
Requests hitting CDN at T0	10,000 (+ refreshes)
Dropped by WAF/rate limit (bots, > 5 req/s)	~ several hundred
Admitted by waiting room in first second	up to 20,000/s → all 10k within ~0.5 s
Lua gate → <code>OK</code>	100
Lua gate → <code>SOLD_OUT</code>	~9,700
Lua gate → <code>DUPLICATE</code> / <code>LIMIT_REACHED</code> (≈2% duplicates)	~200
DB transactions on hot row	≈ 100 (≈1 ms each → trivial)
Successful reservations	100
Payment failures (5%) → released units	~5 → re-offered to waiting room → ~5 more reservations
Final SOLD	100 (never more)

9. When Inventory Reaches Zero

1. Lua gate returns `SOLD_OUT` in < 1 ms for every request.
2. Inventory publishes `inventory.sold_out` → Sale Service sets sale status `SOLD_OUT`, CDN page badge purged/updated, **waiting room closes the queue** and tells queued users "Sold out – join waitlist".
3. If units are released later (payment failure/expiry), `inventory.restocked` reopens admission for that many units.

10. Consistency Guarantees Summary

Data	Consistency	Mechanism
Inventory counts	Strong (linearizable per SKU row)	Single-row ACID update + CHECKs
Reservation status	Strong	Guarded UPDATES
Redis stock counter	Eventually consistent with DB (≤ 10 s), never higher than DB	Compensation + reconciler
Displayed stock badge	Eventual (≤ 1 s)	Cached

6 Payment & Order Workflow Design

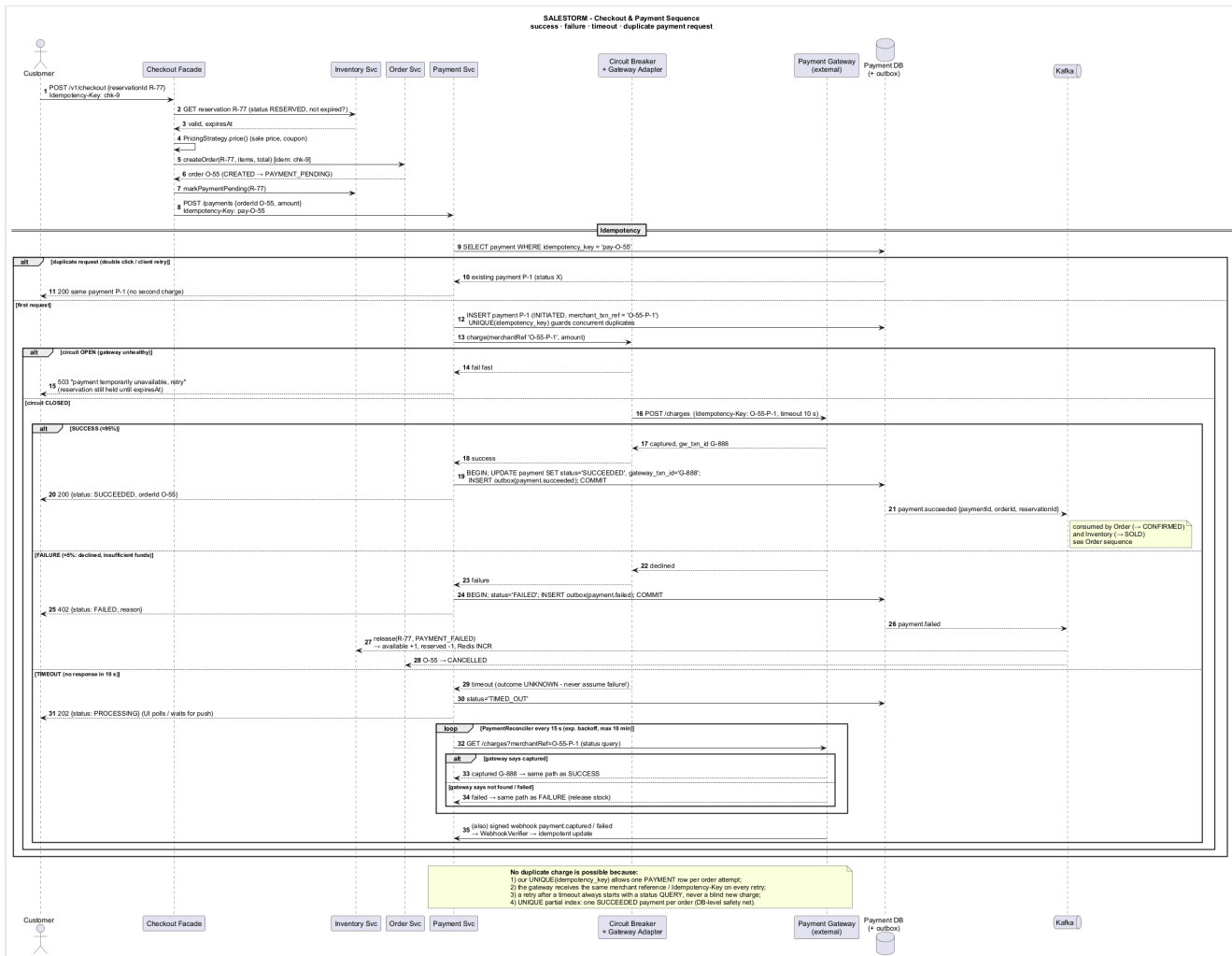


Figure 6.1 – Checkout & Payment Sequence (success, failure, timeout, duplicate)

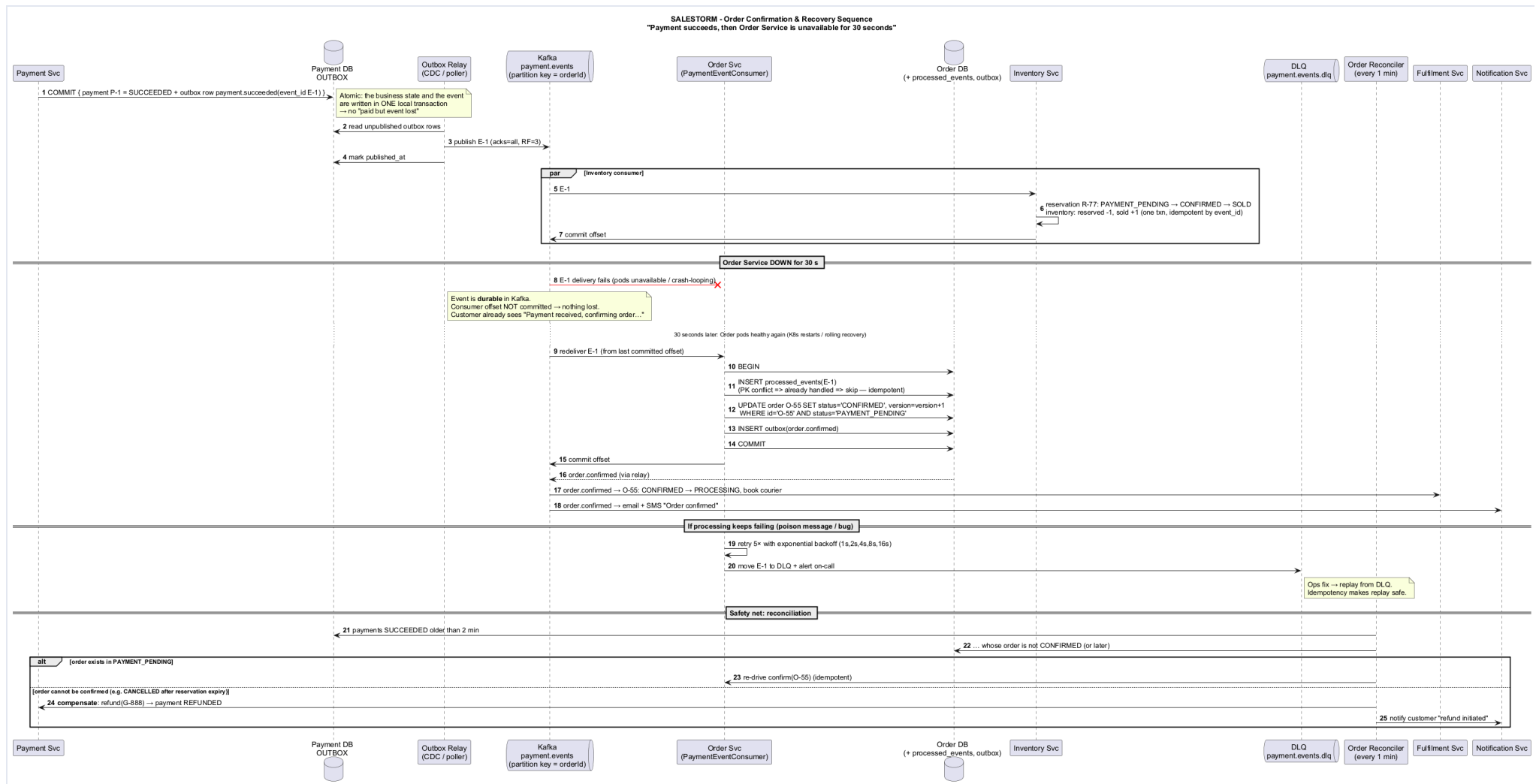


Figure 6.2 – Order Confirmation & Recovery Sequence (Order Service down for 30 s)

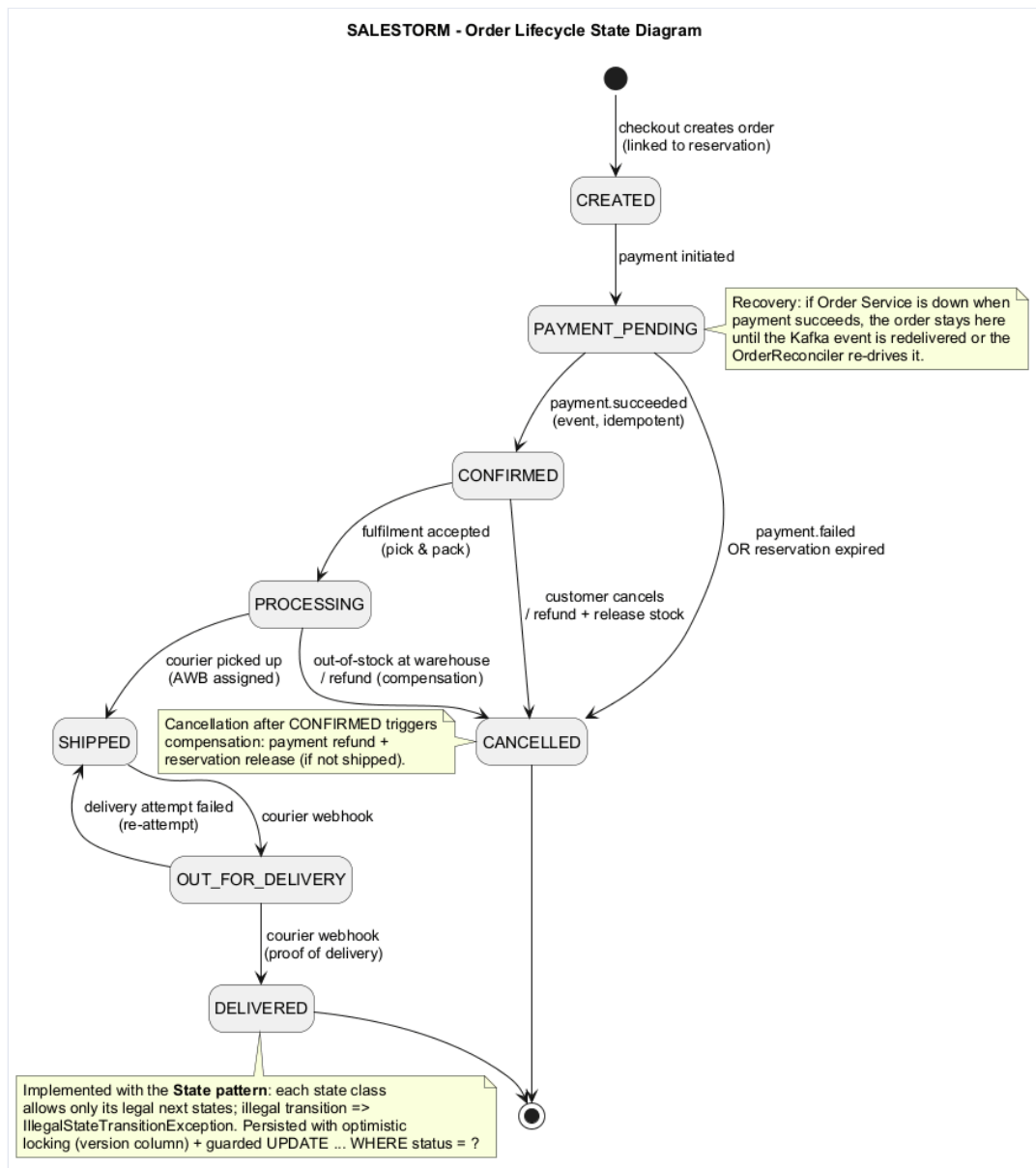


Figure 6.3 – Order Lifecycle State Diagram

1. Workflow Style: Orchestrated Saga + Choreographed Events

- **Checkout Service** orchestrates the *synchronous* part the user waits for: validate reservation → price → create order (`PAYMENT_PENDING`) → initiate payment.
- After the payment outcome is known, the flow is **event-driven** (choreography over Kafka): `payment.succeeded` / `payment.failed` → Inventory (sell / release) and Order (confirm / cancel) → Fulfilment, Notification.
- No 2-phase commit. Each step is a **local transaction + outbox event**; failures are handled by **retries, idempotent consumers, reconciliation and compensation**.

2. Payment Design

2.1 Idempotency (no duplicate charges)

Layer	Mechanism
Client → Payment API	Idempotency-Key header (= <code>pay-<orderId>-<attemptNo></code>). Stored in <code>PAYMENT.idempotency_key</code> UNIQUE. Repeat → return stored response.
Payment → Gateway	Unique <code>merchant_txn_ref</code> sent as gateway idempotency key / order reference. Gateway dedupes on its side.
Retry after timeout	Always query status first (<code>GET /charges?ref=</code>); never fire a blind second charge.
DB safety net	Partial unique index: one <code>SUCCEEDED</code> payment per <code>order_id</code> .
Webhooks	Verified via HMAC signature; processed idempotently by <code>gateway_txn_id</code> .

2.2 Payment State Machine

```
INITIATED → PENDING → SUCCEEDED → (REFUNDED)
                        ↘ FAILED
                        ↘ TIMED_OUT → (reconciler) → SUCCEEDED | FAILED
```

2.3 Scenarios

Scenario	Design response
Payment succeeds	Commit <code>SUCCEEDED</code> + outbox <code>payment.succeeded</code> atomically → Kafka → Order <code>CONFIRMED</code> , Reservation <code>SOLD</code> , Notification sent.
Payment fails	Commit <code>FAILED</code> + outbox <code>payment.failed</code> → Inventory releases unit (DB + Redis) → Order <code>CANCELLED</code> → customer notified; customer may retry with a new attempt while reservation is still valid.
Payment times out	Mark <code>TIMED_OUT</code> (outcome unknown). Customer sees "Processing". <code>PaymentReconciler</code> queries gateway with backoff (15 s → ... → 10 min) and also listens to webhooks. Resolves to <code>SUCCEEDED</code> or <code>FAILED</code> path. Reservation is not released while payment is unresolved.
Duplicate payment request	Same idempotency key → stored result returned; gateway receives the same merchant reference → no second charge.
Payment succeeds, Order Service fails	Event sits durably in Kafka; consumer offset not committed; redelivered after recovery; idempotent processing (<code>processed_events</code>). After 5 retries → DLQ + alert. <code>OrderReconciler</code> re-drives orders stuck in <code>PAYMENT_PENDING</code> with a <code>SUCCEEDED</code> payment. If the order truly cannot be fulfilled → compensating refund .

2.4 Timeouts, Retries, Circuit Breaker

Call	Timeout	Retry policy	Circuit breaker
Checkout → Inventory/Order/Payment (internal)	500 ms / 1 s / 12 s	2 retries, exp. backoff + jitter, only with same idempotency key	Yes (Resilience4j), 50% failure over 20 calls → open 30 s

Call	Timeout	Retry policy	Circuit breaker
Payment → Gateway charge	10 s	No blind retry ; status query instead	Yes: 50% failures or slow calls > 5 s over 20 calls → OPEN 30 s → HALF_OPEN 5 trial calls
Payment → Gateway status query	3 s	10 attempts, exp. backoff up to 10 min	Shared breaker
Kafka consumers	–	5 retries (1,2,4,8,16 s) → DLQ	–
Notification → providers	5 s	3 retries → DLQ	Per provider

Circuit OPEN behaviour: fail fast with `503 PAYMENT_UNAVAILABLE` ; reservation stays held until expiry so the customer can retry; if a **secondary provider** is configured, `PaymentGatewayFactory` routes new attempts to it (Strategy + Adapter).

3. Order Design

3.1 States & Transitions

`CREATED` → `PAYMENT_PENDING` → `CONFIRMED` → `PROCESSING` → `SHIPPED` → `OUT_FOR_DELIVERY` → `DELIVERED` plus `CANCELLED` from `PAYMENT_PENDING`, `CONFIRMED`, `PROCESSING`.

From	Event	To	Side effects
–	checkout	CREATED	link reservation
CREATED	payment initiated	PAYMENT_PENDING	–
PAYMENT_PENDING	payment.succeeded	CONFIRMED	outbox <code>order.confirmed</code>
PAYMENT_PENDING	payment.failed / reservation.expired	CANCELLED	notify
CONFIRMED	fulfilment accepted	PROCESSING	–
CONFIRMED / PROCESSING	cancel / stock issue	CANCELLED	refund + release (compensation)
PROCESSING	courier pickup	SHIPPED	AWB stored
SHIPPED	courier webhook	OUT_FOR_DELIVERY	notify
OUT_FOR_DELIVERY	delivered webhook	DELIVERED	notify
OUT_FOR_DELIVERY	failed attempt	SHIPPED	re-attempt

Implemented with the **State pattern** (`OrderState` classes); persisted with guarded `UPDATE ... WHERE status = :expected AND version = :v`.

3.2 Recovery Paths

Failure	Detection	Recovery
Order Service down (30 s)	Kafka consumer lag alert	Auto redelivery; idempotent consumer

Failure	Detection	Recovery
Poison message	5 failed retries	DLQ + alert; replay after fix
Event lost between DB and Kafka	Impossible by design (outbox)	Outbox relay retries unpublished rows
Paid but order never confirmed	OrderReconciler every 1 min	Re-drive confirm, else refund
Order confirmed but reservation already released (late payment after expiry)	Inventory confirm affects 0 rows	Try re-reserving a unit; if none → cancel + refund + apology notification

4. Practical Test Case Walkthrough

Stock 100 · 10,000 users · 95% success · 5% failures · 2% duplicates · Order Service down 30 s

- 10,000 requests → ~200 duplicates answered from idempotency store → 100 reservations, the rest **SOLD_OUT**.
- 100 checkouts → 100 orders **PAYMENT_PENDING**.
- ~95 payments succeed, ~5 fail → 5 units released → waiting room admits next 5 buyers → ~5 more reservations → (95% of those succeed ...) → converges to **100 SOLD**.
- Order Service down for 30 s exactly while **payment.succeeded** events arrive → events accumulate in Kafka (lag ≈ 95 messages) → service recovers → all consumed in < 1 s → 100 orders **CONFIRMED**. **Zero lost, zero duplicate orders.**
- Duplicate pay clicks (2%) → same payment returned; gateway sees same merchant ref → **0 double charges**.

7 Low-Level Design: Class Model

The class diagram covers the three critical modules required by the brief — Inventory, Payment and Order — plus the Checkout facade and shared infrastructure abstractions. Interfaces are shown in italics; stereotypes mark the design pattern each class plays.

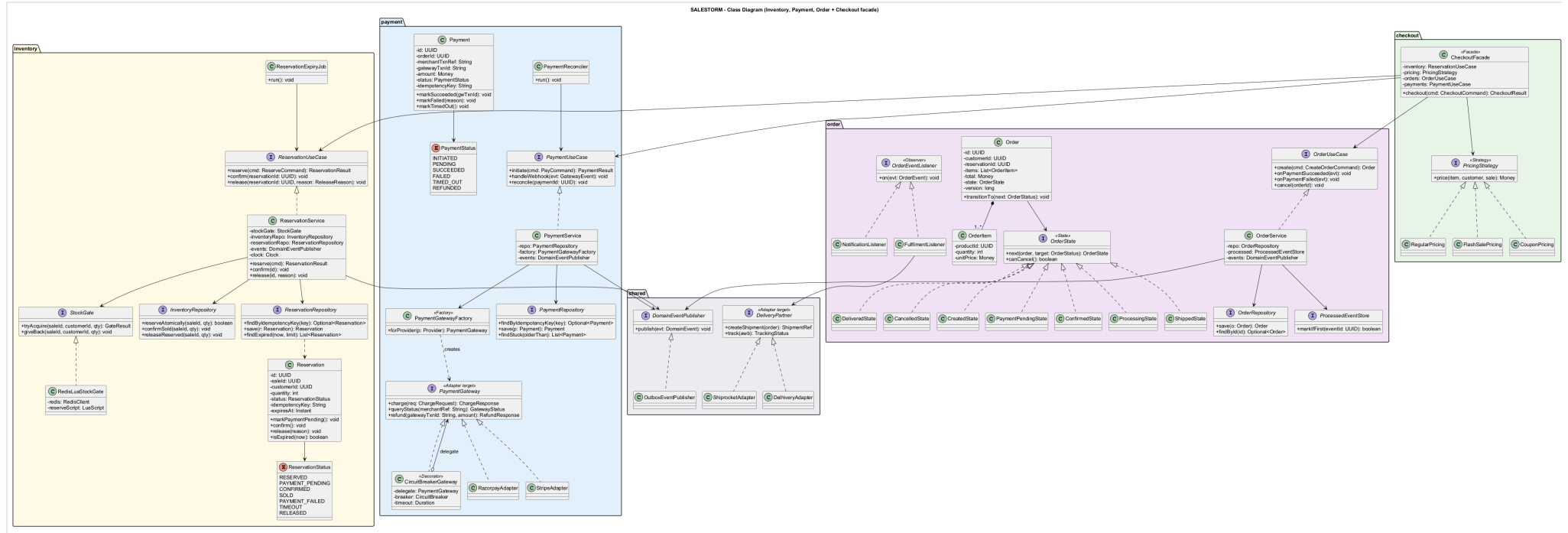


Figure 7.1 – Class Diagram (Inventory, Payment, Order, Checkout, shared)

Where validation, idempotency and error handling live

Concern	Location
Authentication, rate limits, schema validation	API Gateway (edge) + controller bean validation
Business validation (sale live, limit, reservation not expired)	Use-case services: ReservationService, CheckoutFacade, OrderService
Idempotency of API requests	IdempotencyFilter + UNIQUE <code>idempotency_key</code> columns
Idempotency of events	ProcessedEventStore + guarded <code>UPDATE ... WHERE status = ?</code>
Illegal state transitions	OrderState / Reservation domain methods throw <code>IllegalStateException</code>
External failures	CircuitBreakerGateway decorator, timeouts, reconcilers, DLQ
Error responses	Global exception handler → RFC 7807 Problem JSON with stable error codes

8 SOLID Principles & Design Patterns

Principle	Where it appears (actual classes)	What it buys us
S – Single Responsibility	<code>ReservationService</code> (reserve/confirm/release only) · <code>ReservationExpiryJob</code> (expiry only) · <code>PaymentService</code> (payment lifecycle) · <code>PaymentReconciler</code> (resolving unknown outcomes) · <code>OrderService</code> (order lifecycle) · <code>NotificationListener</code> (messaging) · <code>WebhookVerifier</code> (signature check) · <code>OutboxEventPublisher</code> (event persistence)	Each class has one reason to change. Changing SMS provider never touches payment code; changing expiry policy never touches the reserve path.
O – Open/Closed	<code>PricingStrategy</code> → <code>FlashSalePricing</code> , <code>CouponPricing</code> , <code>RegularPricing</code> · <code>PaymentGateway</code> → <code>RazorpayAdapter</code> , <code>StripeAdapter</code> · <code>DeliveryPartner</code> → <code>DelhiveryAdapter</code> , <code>ShiprocketAdapter</code> · <code>OrderEventListener</code> observers	New pricing rule / payment provider / courier = new class , registered in config/factory. <code>CheckoutFacade</code> , <code>PaymentService</code> , <code>FulfilmentListener</code> are not modified.
L – Liskov Substitution	Any <code>PaymentGateway</code> (incl. <code>CircuitBreakerGateway</code> decorator) is usable wherever <code>PaymentGateway</code> is expected; every <code>OrderState</code> honours <code>next()</code> / <code>canCancel()</code> contract; <code>RedisLuaStockGate</code> vs a test <code>InMemoryStockGate</code>	Contract rules: adapters must be idempotent on <code>merchantRef</code> , must map provider errors to our <code>GatewayStatus</code> enum, must never throw provider-specific exceptions. Substitution doesn't break callers or tests.
I – Interface Segregation	Small focused interfaces: <code>StockGate</code> (2 methods), <code>ReservationRepository</code> , <code>InventoryRepository</code> (separate), <code>PaymentUseCase</code> vs <code>PaymentGateway</code> (business API vs provider API), <code>ProcessedEventStore</code> (1 method), <code>DomainEventPublisher</code> (1 method)	No class depends on methods it doesn't use. <code>ReservationExpiryJob</code> only sees <code>ReservationUseCase</code> , not the repositories. <code>Webhook</code> controller doesn't see refund APIs.
D – Dependency Inversion	<code>ReservationService</code> depends on <code>StockGate</code> , <code>InventoryRepository</code> , <code>DomainEventPublisher</code> (abstractions), not <code>Redis/JDBC/Kafka</code> classes. <code>CheckoutFacade</code> depends on <code>ReservationUseCase</code> , <code>OrderUseCase</code> , <code>PaymentUseCase</code> , <code>PricingStrategy</code> . <code>PaymentService</code> depends on <code>PaymentGateway</code> , obtained from <code>PaymentGatewayFactory</code> . Concrete wiring done by Spring DI.	High-level business policy is independent of infrastructure → swap <code>Redis</code> for another gate, <code>Kafka</code> for <code>SQS</code> , <code>Razorpay</code> for <code>Stripe</code> without touching business logic; unit tests use in-memory fakes.

Example – adding a new payment provider ("PayU") with minimal change

1. Create `PayUAdapter` implements `PaymentGateway` (maps `PayU` API ↔ our `ChargeRequest/ChargeResponse/GatewayStatus`).
2. Register in `PaymentGatewayFactory` (`Provider.PAYU` → `PayUAdapter`), add config + secrets.
3. **No change** to `PaymentService`, `CheckoutFacade`, `PaymentReconciler`, events, DB schema.

Example – new pricing strategy ("Buy-2-Get-10%-off")

class `BundlePricing` implements `PricingStrategy` + register in `PricingEngine` strategy map keyed by `sale.pricingType`. `CheckoutFacade` unchanged.

Example – new delivery partner

class `BlueDartAdapter` implements `DeliveryPartner` + routing rule in `FulfilmentListener` (Strategy by `pincode/weight`). Order and Payment untouched.

Code sketch (Java) – DIP + OCP in the reservation path

```
public interface StockGate {
    GateResult tryAcquire(UUID saleId, UUID customerId, int qty, String idemKey);
    void giveBack(UUID saleId, UUID customerId, int qty);
}

@Service
public class ReservationService implements ReservationUseCase {
    private final StockGate gate; // abstraction (DIP)
    private final InventoryRepository inventory; // abstraction (DIP)
    private final ReservationRepository reservations; // focused interface (ISP)
    private final DomainEventPublisher events; // outbox-backed

    @Transactional
    public ReservationResult reserve(ReserveCommand cmd) {
        var existing = reservations.findByIdempotencyKey(cmd.idemKey());
        if (existing.isPresent()) return ReservationResult.replay(existing.get());

        var gateResult = gate.tryAcquire(cmd.saleId(), cmd.customerId(), cmd.qty(), cmd.idemKey());
        if (!gateResult.ok()) return ReservationResult.rejected(gateResult.reason());

        try {
            if (!inventory.reserveAtomically(cmd.saleId(), cmd.qty())) { // conditional UPDATE
                gate.giveBack(cmd.saleId(), cmd.customerId(), cmd.qty());
                return ReservationResult.rejected(Reason.SOLD_OUT);
            }
            var r = reservations.save(Reservation.create(cmd, clock.instant().plus(holdTime)));
            events.publish(new ReservationCreated(r)); // same transaction
            return ReservationResult.created(r);
        } catch (DuplicateKeyException e) {
            gate.giveBack(cmd.saleId(), cmd.customerId(), cmd.qty());
            return
                ReservationResult.replay(reservations.findByIdempotencyKey(cmd.idemKey()).orElseThrow());
        }
    }
}
```

Design pattern mapping

1. Object-Oriented (GoF) Patterns

Pattern	Where (classes)	Problem it solves	Trade-off introduced
Strategy	PricingStrategy → FlashSalePricing, CouponPricing, RegularPricing; payment-method routing; courier selection by pincode	Pricing rules change every campaign; avoid if/else chains in checkout	More classes; strategy selection logic must be configured & tested
Factory	PaymentGatewayFactory.forProvider()	Create correct gateway adapter (+ wrap in circuit breaker) per provider/config without callers knowing concrete classes	One more indirection; factory must be updated when adding providers
State	OrderState → CreatedState, PaymentPendingState, ConfirmedState, ..., CancelledState; same idea for Reservation	Enforce legal transitions only ; avoid scattered switch(status) and illegal jumps (e.g. DELIVERED → PAYMENT_PENDING)	Many small classes; state must still be persisted as enum + guarded UPDATE
Observer	OrderEventListener → NotificationListener, FulfilmentListener (in-process); Kafka consumers (distributed)	Many parties react to order events; producer shouldn't know them	Harder to follow control flow; need tracing and idempotent listeners
Adapter	RazorpayAdapter, StripeAdapter (→ PaymentGateway); DelhiveryAdapter, ShiprocketAdapter (→ DeliveryPartner)	External APIs have different formats/errors; isolate them behind our interface	Lowest-common-denominator interface may hide provider-specific features
Facade	CheckoutFacade	Client would otherwise call Inventory, Pricing, Order, Payment separately and handle partial failures	Facade can become a "god service" – kept thin, orchestration only
Repository	InventoryRepository, ReservationRepository, PaymentRepository, OrderRepository	Decouple domain logic from SQL/ORM; enable in-memory fakes in tests	Hides SQL – critical queries (conditional UPDATE) must remain explicit, not ORM-generated
Decorator	CircuitBreakerGateway wraps any PaymentGateway	Add timeout/retry/circuit-breaking without changing adapters	Stacking decorators can obscure behaviour; configured centrally

Pattern	Where (classes)	Problem it solves	Trade-off introduced
Template Method <i>(optional)</i>	AbstractScheduledJob for ReservationExpiryJob , PaymentReconciler , OrderReconciler (lock → fetch batch → process → metrics)	Shared job skeleton with leader election & batching	Inheritance coupling

2. Resilience & Distributed-System Patterns

Pattern	Where	Problem solved	Trade-off
Circuit Breaker	Payment → Gateway; Checkout → internal services; Notification → providers	Stop hammering a failing dependency, fail fast, give it time to recover; prevent cascading failure / thread exhaustion	Some requests rejected while OPEN even if dependency recovered; thresholds need tuning
Retry with exponential backoff + jitter	Internal calls, Kafka consumers, reconcilers	Transient failures	Can amplify load (retry storm) → bounded retries + only idempotent operations
Timeout	Every remote call	Hung calls tie up threads	Too short → false failures (payment uses status query to handle that)
Idempotency Key / Idempotent Receiver	Reservation, checkout, payment APIs; all Kafka consumers (processed_event)	Duplicate requests/messages must not duplicate business effects	Storage for keys; TTL management
Transactional Outbox	Inventory, Payment, Order DBs → Kafka	Avoid dual-write (DB committed but event lost, or vice versa)	Extra table + relay process; slight event latency (~100 ms)
Saga (orchestration + choreography)	Checkout orchestrates sync steps; events choreograph post-payment	Long-running business transaction across services without 2PC	Eventual consistency; need compensating actions
Compensating Transaction	Release reservation on payment failure; refund when paid order can't be fulfilled	Undo a completed step in a distributed flow	Business must accept "undo" semantics (refund, apology)
Dead Letter Queue	*.dlq topics	Poison messages shouldn't block partitions	Requires ops tooling & replay process
Bulkhead	Separate thread pools / pods for reserve path vs browse; separate DB per service	Failure or overload in one area doesn't sink others	Lower resource utilisation
Rate Limiter (Token Bucket)	API Gateway (per user / per IP)	Abuse, bots, retry storms	Legitimate bursts may be throttled
Queue-Based Load Leveling	Virtual waiting room; Kafka	Smooth spikes to a rate the backend can handle	Added wait time for users

Pattern	Where	Problem solved	Trade-off
Cache-Aside	Product/sale pages in Redis + CDN	Read scalability	Staleness (≤ 30 s product, ≤ 1 s stock badge)
Database per Service	Inventory/Order/Payment DBs	Independent scaling & failure isolation	Cross-service queries need events/APIs
Reconciliation (anti-entropy)	StockReconciler , PaymentReconciler , OrderReconciler	Repair any drift that slipped through	Periodic extra load; must itself be idempotent

3. How Patterns Combine on the Critical Path

```

Customer → [Rate Limiter] → [Queue-Based Load Leveling: Waiting Room]
    → ReservationController → [Idempotent Receiver]
    → ReservationService → StockGate [Strategy-able, Redis Lua]
                        → InventoryRepository [Repository] (atomic UPDATE)
                        → DomainEventPublisher [Transactional Outbox]
    → CheckoutFacade [Facade + Saga orchestrator]
        → PricingStrategy [Strategy]
        → PaymentService → PaymentGatewayFactory [Factory]
                        → CircuitBreakerGateway [Decorator + Circuit Breaker]
                        → RazorpayAdapter [Adapter]
    → Kafka [Observer / choreography] → OrderService [State] → FulfilmentListener →
DeliveryPartner [Adapter]
                                → failures → [Retry] → [DLQ] → [Reconciliation /
Compensation]

```

9 Database Design

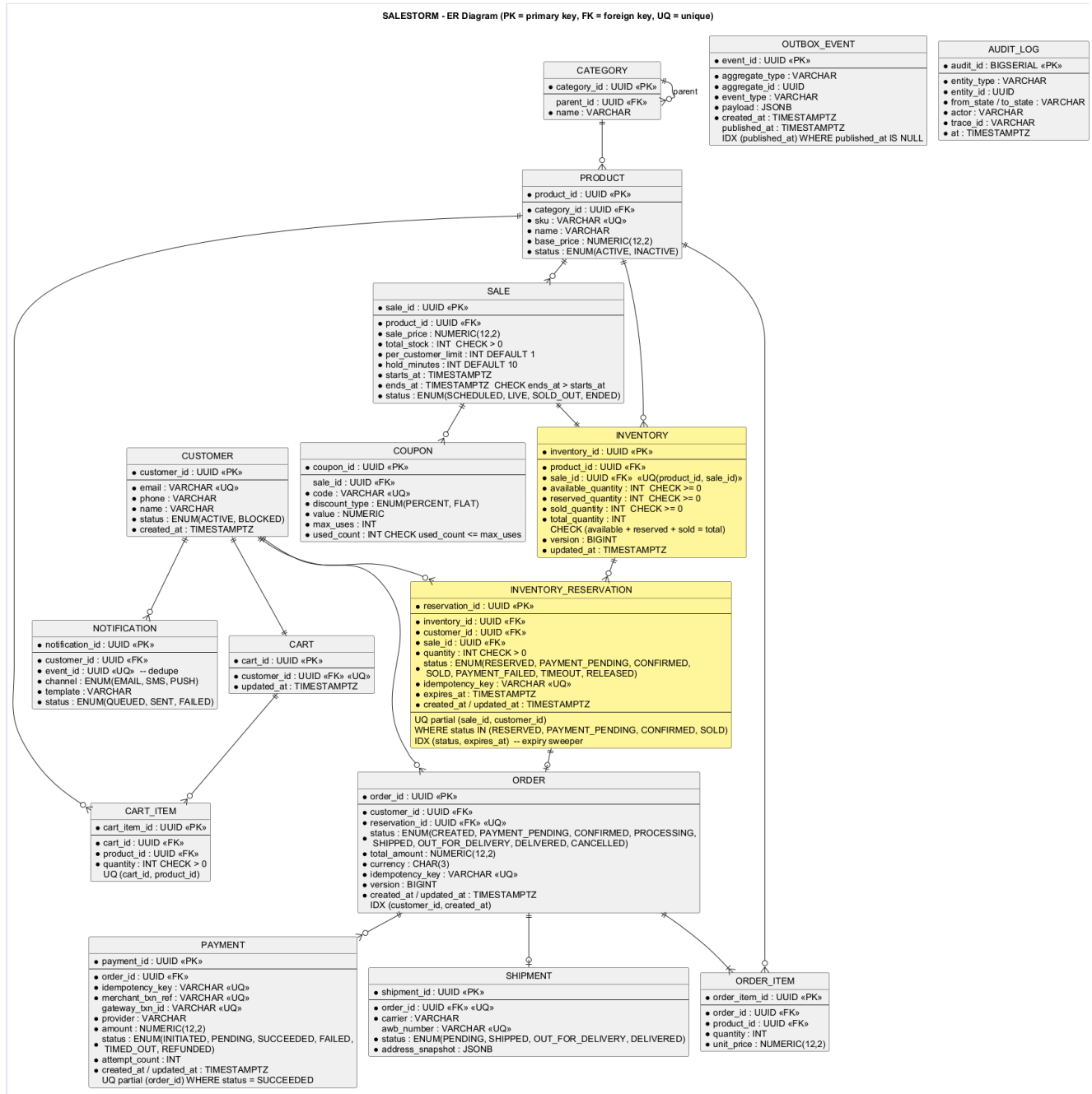


Figure 9.1 – ER Diagram

1. Database Choice & Ownership

Service	Store	Tables owned	Why
Catalog / Sale	PostgreSQL + OpenSearch (search) + Redis (cache)	PRODUCT, CATEGORY, SALE, DEAL, COUPON	Read-heavy, relational; search index for discovery
Cart	Redis (primary) + PostgreSQL snapshot	CART, CART_ITEM	Fast mutable state; loss-tolerant

Service	Store	Tables owned	Why
Inventory	PostgreSQL + Redis gate	INVENTORY, INVENTORY_RESERVATION, OUTBOX_EVENT	ACID + CHECK constraints for correctness
Order	PostgreSQL	ORDER, ORDER_ITEM, PROCESSED_EVENT, OUTBOX_EVENT	Transactions, state machine
Payment	PostgreSQL	PAYMENT, PAYMENT_ATTEMPT, OUTBOX_EVENT	Money → strongest guarantees
Fulfilment	PostgreSQL	SHIPMENT	Relational
Notification	PostgreSQL / DynamoDB	NOTIFICATION	High write volume, simple access
All	Append-only	AUDIT_LOG	Traceability

Database-per-service: no service reads another service's tables; cross-service references (e.g. `order.reservation_id`) are **logical FKs** validated via API/events, physical FKs only inside one service's DB.

2. DDL (core tables)

```
-- ===== Catalog =====
CREATE TABLE customer (
  customer_id UUID PRIMARY KEY,
  email        VARCHAR(255) NOT NULL UNIQUE,
  phone        VARCHAR(20)  NOT NULL,
  name         VARCHAR(120) NOT NULL,
  status       VARCHAR(10)  NOT NULL DEFAULT 'ACTIVE' CHECK (status IN ('ACTIVE','BLOCKED')),
  created_at   TIMESTAMPTZ  NOT NULL DEFAULT now()
);

CREATE TABLE category (
  category_id UUID PRIMARY KEY,
  parent_id   UUID REFERENCES category(category_id),
  name        VARCHAR(100) NOT NULL
);

CREATE TABLE product (
  product_id  UUID PRIMARY KEY,
  category_id UUID NOT NULL REFERENCES category(category_id),
  sku         VARCHAR(64) NOT NULL UNIQUE,
  name        VARCHAR(200) NOT NULL,
  base_price  NUMERIC(12,2) NOT NULL CHECK (base_price >= 0),
  status      VARCHAR(10) NOT NULL DEFAULT 'ACTIVE'
);
CREATE INDEX ix_product_category ON product(category_id);

CREATE TABLE sale (
  sale_id      UUID PRIMARY KEY,
  product_id   UUID NOT NULL REFERENCES product(product_id),
  sale_price   NUMERIC(12,2) NOT NULL CHECK (sale_price >= 0),
  total_stock  INT NOT NULL CHECK (total_stock > 0),
  per_customer_limit INT NOT NULL DEFAULT 1 CHECK (per_customer_limit > 0),
  hold_minutes INT NOT NULL DEFAULT 10,
  starts_at    TIMESTAMPTZ NOT NULL,
  ends_at      TIMESTAMPTZ NOT NULL,
  status       VARCHAR(12) NOT NULL DEFAULT 'SCHEDULED',
  CHECK (ends_at > starts_at)
);
CREATE INDEX ix_sale_status_start ON sale(status, starts_at);

CREATE TABLE coupon (
  coupon_id    UUID PRIMARY KEY,
  sale_id      UUID REFERENCES sale(sale_id),
  code         VARCHAR(32) NOT NULL UNIQUE,
  discount_type VARCHAR(8) NOT NULL CHECK (discount_type IN ('PERCENT','FLAT')),
  value        NUMERIC(10,2) NOT NULL CHECK (value > 0),
  max_uses     INT NOT NULL,
  used_count   INT NOT NULL DEFAULT 0,
  CHECK (used_count <= max_uses)
);

-- ===== Inventory (see Concurrency_and_Inventory_Design.md for full detail) =====
-- inventory, inventory_reservation with CHECK (available >= 0) and conservation invariant

-- ===== Order =====
CREATE TABLE "order" (
  order_id      UUID PRIMARY KEY,
  customer_id   UUID NOT NULL,
  reservation_id UUID NOT NULL UNIQUE,           -- one order per reservation
  status        VARCHAR(20) NOT NULL,
  total_amount  NUMERIC(12,2) NOT NULL CHECK (total_amount >= 0),
```

```

currency          CHAR(3) NOT NULL DEFAULT 'INR',
idempotency_key   VARCHAR(64) NOT NULL UNIQUE,
version           BIGINT NOT NULL DEFAULT 0,
created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
CHECK (status IN ('CREATED','PAYMENT_PENDING','CONFIRMED','PROCESSING',
                  'SHIPPED','OUT_FOR_DELIVERY','DELIVERED','CANCELLED'))
);
CREATE INDEX ix_order_customer ON "order"(customer_id, created_at DESC);
CREATE INDEX ix_order_status ON "order"(status, updated_at); -- reconciler

CREATE TABLE order_item (
  order_item_id UUID PRIMARY KEY,
  order_id       UUID NOT NULL REFERENCES "order"(order_id) ON DELETE CASCADE,
  product_id     UUID NOT NULL,
  quantity       INT NOT NULL CHECK (quantity > 0),
  unit_price     NUMERIC(12,2) NOT NULL
);

CREATE TABLE processed_event (          -- idempotent consumers
  event_id       UUID PRIMARY KEY,
  consumer       VARCHAR(50) NOT NULL,
  processed_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ===== Payment =====
CREATE TABLE payment (
  payment_id     UUID PRIMARY KEY,
  order_id       UUID NOT NULL,
  idempotency_key VARCHAR(64) NOT NULL UNIQUE,
  merchant_txn_ref VARCHAR(64) NOT NULL UNIQUE,
  gateway_txn_id VARCHAR(64) UNIQUE,
  provider       VARCHAR(20) NOT NULL,
  amount         NUMERIC(12,2) NOT NULL CHECK (amount > 0),
  status         VARCHAR(12) NOT NULL,
  attempt_count  INT NOT NULL DEFAULT 0,
  created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX uq_one_success_per_order ON payment(order_id) WHERE status = 'SUCCEEDED';
CREATE INDEX ix_payment_stuck ON payment(status, updated_at) WHERE status IN ('PENDING','TIMED_OUT');

-- ===== Fulfilment / Notification =====
CREATE TABLE shipment (
  shipment_id    UUID PRIMARY KEY,
  order_id       UUID NOT NULL UNIQUE,
  carrier        VARCHAR(30) NOT NULL,
  awb_number     VARCHAR(40) UNIQUE,
  status         VARCHAR(20) NOT NULL,
  address_snapshot JSONB NOT NULL
);

CREATE TABLE notification (
  notification_id UUID PRIMARY KEY,
  customer_id     UUID NOT NULL,
  event_id        UUID NOT NULL UNIQUE,          -- never send the same notification twice
  channel         VARCHAR(5) NOT NULL,
  template        VARCHAR(50) NOT NULL,
  status          VARCHAR(8) NOT NULL
);

-- ===== Cross-cutting =====
CREATE TABLE outbox_event (
  event_id       UUID PRIMARY KEY,

```

```

aggregate_type VARCHAR(30) NOT NULL,
aggregate_id    UUID NOT NULL,
event_type     VARCHAR(50) NOT NULL,
payload        JSONB NOT NULL,
created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
published_at   TIMESTAMPTZ
);
CREATE INDEX ix_outbox_unpublished ON outbox_event(created_at) WHERE published_at IS NULL;

CREATE TABLE audit_log (
  audit_id    BIGSERIAL PRIMARY KEY,
  entity_type VARCHAR(30) NOT NULL,
  entity_id   UUID NOT NULL,
  from_state  VARCHAR(20),
  to_state    VARCHAR(20) NOT NULL,
  actor       VARCHAR(60) NOT NULL,
  trace_id    VARCHAR(40),
  at          TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ix_audit_entity ON audit_log(entity_type, entity_id, at);

```

3. Design Checklist (brief requirement)

Requirement	How addressed
Primary keys	UUID v7 (time-ordered → index-friendly, globally unique, generated by app → safe for idempotent inserts)
Foreign keys	Inside each service DB; logical references across services
Indexes	Lookup by customer, status+time for reconcilers/sweepers, partial indexes for hot subsets
Constraints	CHECK non-negative quantities, conservation invariant, enum checks, UNIQUE idempotency keys, partial unique (one active reservation per customer, one successful payment per order)
Transaction boundaries	One aggregate per transaction per service: (inventory row + reservation + outbox), (payment + outbox), (order + processed_event + outbox)
Concurrency control	Atomic conditional UPDATE (inventory), guarded status UPDATES, optimistic version for orders, SKIP LOCKED for job workers
Consistency	Strong within a service; eventual across services via outbox + Kafka
Audit	audit_log row per state transition with trace_id ; created_at/updated_at on all tables

4. Scaling the Data Layer

- **Read replicas** for catalog/order history; **writes** to primary only.
- **Partitioning:** order , payment , audit_log range-partitioned by month; outbox_event purged after publish + 7 days.
- **Sharding (future):** inventory by sale_id / sku_id , orders by customer_id (Citus / Vitess-style).
- **Connection pooling:** PgBouncer (transaction mode) per DB.
- **HA:** primary + synchronous standby (RPO 0) in another AZ; automatic failover ≈ 30 s.

5. SQL vs NoSQL Decision (summary – see ADR-002)

Relational (PostgreSQL) for inventory/order/payment because we need **multi-row ACID transactions, CHECK and UNIQUE constraints** — exactly the tools that make overselling and duplicate payments *impossible* rather than *unlikely*. NoSQL/Redis used where we need speed and can tolerate loss (cache, cart, gate, rate limiting).

10 API & Event Design

1. Conventions

Aspect	Rule
Base URL	<code>https://api.salestorm.com/v1</code>
Transport	HTTPS only (TLS 1.2+); HSTS
Auth	Authorization: Bearer <JWT> (OIDC access token, 15 min). Admin APIs need <code>role=admin</code> scope. Webhooks: HMAC signature header, no JWT.
Idempotency	Idempotency-Key: <uuid> required on all POSTs that create business transactions (<code>/reservations</code> , <code>/checkout</code> , <code>/payments</code> , <code>/orders/{id}/cancel</code>). Same key + same body → same response (stored 24 h). Same key + different body → <code>422 IDEMPOTENCY_KEY_REUSED</code> .
Tracing	X-Request-ID (generated at gateway if absent) + W3C <code>traceparent</code> propagated everywhere
Rate limits	Per user 5 req/s on buy/checkout, 50 req/s per IP; headers <code>X-RateLimit-Remaining</code> , <code>Retry-After</code>
Errors	RFC 7807 Problem JSON: { <code>"type"</code> , <code>"title"</code> , <code>"status"</code> , <code>"code"</code> , <code>"detail"</code> , <code>"traceId"</code> }
Versioning	URI version <code>/v1</code> ; additive changes only within a version

Standard error codes

HTTP	Code	Meaning
400	<code>VALIDATION_ERROR</code>	Bad input
401	<code>UNAUTHENTICATED</code>	Missing/invalid token
403	<code>FORBIDDEN</code> / <code>ADMISSION_REQUIRED</code>	No permission / no waiting-room token
404	<code>NOT_FOUND</code>	Unknown resource
409	<code>SOLD_OUT</code> / <code>LIMIT_REACHED</code> / <code>RESERVATION_EXPIRED</code> / <code>INVALID_STATE</code>	Business conflict
402	<code>PAYMENT_FAILED</code>	Declined
422	<code>IDEMPOTENCY_KEY_REUSED</code>	Key reused with different payload
429	<code>RATE_LIMITED</code>	Too many requests (+ <code>Retry-After</code>)
503	<code>PAYMENT_UNAVAILABLE</code> / <code>SERVICE_UNAVAILABLE</code>	Circuit open / dependency down (+ <code>Retry-After</code>)

2. REST Endpoints

2.1 Discovery & Sale

Method	Endpoint	Auth	Description	Responses
GET	<code>/products?</code> <code>category=&q=&page=</code>	Public	Search/browse (CDN-cached 30 s)	200
GET	<code>/products/{productId}</code>	Public	Product detail	200, 404
GET	<code>/sales/{saleId}</code>	Public	Sale info: price, start/end, status	200, 404
GET	<code>/sales/{saleId}/stock</code>	Public	Approximate stock { "available": 37, "status": "LIVE" } (cached 1 s)	200
POST	<code>/admin/sales</code>	Admin	Create sale (product, stock, price, window, limit)	201, 400

2.2 Cart

Method	Endpoint	Description	Responses
GET	<code>/cart</code>	Current cart	200
POST	<code>/cart/items</code>	{ productId, quantity }	201, 409 LIMIT_REACHED
DELETE	<code>/cart/items/{itemId}</code>	Remove	204

2.3 Waiting Room

Method	Endpoint	Description	Responses
POST	<code>/sales/{saleId}/queue</code>	Join queue	202 { ticketId, position, estimatedWaitSec }
GET	<code>/sales/{saleId}/queue/{ticketId}</code>	Poll	200 { status: WAITING \ ADMITTED \ SOLD_OUT, admissionToken? }

2.4 Reservation (critical)

POST `/sales/{saleId}/reservations`

Headers: Authorization, Idempotency-Key, X-Admission-Token

```
// Request
{ "productId": "p-1", "quantity": 1 }

// 201 Created
{ "reservationId": "r-77", "status": "RESERVED", "quantity": 1,
  "expiresAt": "2026-10-05T10:10:00Z", "checkoutUrl": "/v1/checkout" }

// 200 OK (idempotent replay - same body as original 201, header Idempotent-Replayed: true)

// 409 Conflict
{ "type": "https://salestorm.com/errors/sold-out", "title": "Sold out",
  "status": 409, "code": "SOLD_OUT", "traceId": "4bf92f..." }
```

Method	Endpoint	Description	Responses
GET	/reservations/{reservationId}	Status + expiresAt	200, 404
DELETE	/reservations/{reservationId}	Customer releases hold	204, 409 INVALID_STATE

2.5 Checkout

POST /checkout (Idempotency-Key required)

```
// Request
{ "reservationId": "r-77", "couponCode": "FLASH10",
  "shippingAddressId": "a-3", "paymentMethod": { "type": "UPI" } }

// 201 Created
{ "orderId": "o-55", "orderStatus": "PAYMENT_PENDING",
  "amount": { "value": 8999.00, "currency": "INR" },
  "payment": { "paymentId": "p-1", "status": "PENDING",
    "nextAction": { "type": "REDIRECT", "url": "https://gateway/..." } } }
```

Errors: 409 RESERVATION_EXPIRED , 409 INVALID_STATE , 503 PAYMENT_UNAVAILABLE .

2.6 Payment

Method	Endpoint	Auth	Description	Responses
POST	/payments	JWT (internal: service token)	{ orderId, amount, method } + Idempotency-Key	201 PENDING, 200 replay, 402, 503
GET	/payments/{paymentId}	JWT	Status	200
POST	/webhooks/payments/{provider}	HMAC signature	Gateway callback payment.captured / failed / refunded	200 (always ack after durable store), 401 bad signature
POST	/admin/payments/{paymentId}/refund	Admin	Manual refund	202

2.7 Orders & Tracking

Method	Endpoint	Description	Responses
GET	/orders?status=&page=	Customer's orders	200
GET	/orders/{orderId}	Order detail + status history	200, 404
POST	/orders/{orderId}/cancel	Cancel (if state allows)	202, 409 INVALID_STATE
GET	/orders/{orderId}/tracking	Shipment timeline	200
POST	/webhooks/shipments/{carrier}	Carrier status webhook (HMAC)	200

3. Commands vs Events

Type	Name	Producer	Consumers	Transport
Command (sync)	ReserveStock	API Gateway	Inventory	REST
Command (sync)	CreateOrder	Checkout	Order	REST/gRPC
Command (sync)	InitiatePayment	Checkout	Payment	REST/gRPC
Command (async)	ReleaseReservation	Expiry job / Payment events	Inventory	Kafka
Command (async)	SendNotification	any	Notification	Kafka

4. Event Catalogue (Kafka)

Topic	Event	Owner (producer)	Consumers & responsibility	Key
reservation.events	reservation.created	Inventory	Notification (hold confirmation), Analytics	reservationId
	reservation.released	Inventory	Sale/Waiting Room (re-open admission), Order (cancel if pending)	reservationId
	reservation.expired	Inventory	Order → CANCELLED, Notification	reservationId
inventory.events	inventory.sold_out / inventory.restocked	Inventory	Sale (status), Waiting Room (close/open), CDN purge	saleId
payment.events	payment.succeeded	Payment	Order → CONFIRMED, Inventory → SOLD, Notification	orderId
	payment.failed	Payment	Inventory → RELEASE, Order → CANCELLED, Notification	orderId
	payment.refunded	Payment	Order, Notification	orderId
order.events	order.confirmed	Order	Fulfilment (start), Notification	orderId
	order.cancelled	Order	Payment (refund if paid), Inventory (release), Notification	orderId
shipment.events	shipment.shipped / out_for_delivery / delivered	Fulfilment	Order (state), Notification	orderId
*.dlq	failed messages	consumers	Ops tooling / replay	same

Event envelope (all events)

```
{
  "eventId": "e-1f0c...",           // UUID - consumers dedupe on this
  "eventType": "payment.succeeded",
  "eventVersion": 1,
  "occurredAt": "2026-10-05T10:02:11.402Z",
  "producer": "payment-service",
  "aggregateId": "o-55",
  "traceId": "4bf92f3577b34da6a3ce929d0e0e4736",
  "payload": { "paymentId": "p-1", "orderId": "o-55", "reservationId": "r-77",
               "amount": 8999.00, "currency": "INR", "gatewayTxnId": "G-888" }
}
```

Delivery guarantees

- Producers: **transactional outbox** → at-least-once publish, `acks=all`, idempotent producer enabled.
- Consumers: **at-least-once** consumption + **idempotent handlers** (`processed_event` table / guarded UPDATES) ⇒ effectively-once business effect.
- Ordering: per aggregate via partition key (`orderId` , `reservationId`).
- Schema: JSON Schema / Avro in a schema registry; backward-compatible evolution.

11 Scalability & Reliability

Part A – Scalability

A1. Capacity Plan

Tier	Normal (10k req/s)	Flash sale (500k req/s at edge)	Scaling mechanism
CDN / WAF	8k req/s served at edge	~475k req/s served at edge (95%+)	Global PoPs, pre-warmed, cache sale page 5–30 s
Load balancer	2k req/s	~25k req/s dynamic	Managed ALB, pre-warmed via cloud support ticket
API Gateway	6 pods	60 pods (~500 req/s/pod)	HPA on RPS + scheduled pre-scale T-30 min
Waiting room	3 pods	30 pods; admits 20k/s	Stateless + Redis sorted set
Inventory service	6 pods	30 pods (CPU-light; mostly Redis calls)	HPA
Redis cluster	3 shards	Hot key ≈ 20k–100k ops/s on one shard (Redis handles ~100k+ ops/s single-thread)	Stock bucketing for 50×+ (below)
Inventory DB	–	~100–200 write txns total on hot row	Not a bottleneck thanks to gate
Payment service	3 pods	15 pods; ≈ 100–200 payments	Gateway-bound, not CPU-bound
Kafka	3 brokers	Few thousand events	Partitions = 12 per topic

Pre-scaling, not auto-scaling, is the primary strategy: a flash sale spikes in < 5 s; HPA reacts in 30–90 s. We scale up 30 min before `starts_at` (from Sale config) and scale down 30 min after `SOLD_OUT`.

A2. What Changes at 50× Traffic (10k → 500k concurrent buyers)

Concern	At 10k	At 500k (50×) – change
Edge	CDN + WAF	Same, plus static "sold out" page flip at CDN as soon as <code>inventory.sold_out</code> fires; bot challenge (JS / CAPTCHA by CDN vendor) for suspicious traffic
Admission	Waiting room optional	Waiting room mandatory ; randomised admission among those who arrived in first N seconds (fairness, defeats "fastest bot wins")
Hot key	One <code>stock: {sale}</code> key	Stock bucketing : split 100 units into 10 keys × 10 units on different Redis shards; requests hash to a bucket, on empty bucket try 1–2 others. Spreads load 10×
Gate tier	Redis	Optionally local in-pod token pre-allocation (each Inventory pod leases e.g. 5 units) → zero network hop for rejections
DB	Single primary	Unchanged for writes (still ≤ 100 winners); read replicas for order history

Concern	At 10k	At 500k (50x) – change
Alternative	–	Queue-based serialisation per SKU (Kafka partition, single consumer) if we prefer strict FIFO fairness over instant response
Rate limits	5 req/s/user	Tighter per-IP + per-device fingerprint limits

A3. Bottlenecks Ranked

1. **Hot inventory key/row** → Redis gate + bucketing (DB sees only winners).
2. **Connection storms to DB** → PgBouncer; admission caps concurrency.
3. **Payment gateway rate limits** → only ~100 payments; circuit breaker; secondary provider.
4. **Gateway / LB cold start** → pre-warm.
5. **Kafka consumer lag** → partitions & consumer replicas; lag alerts.

A4. Caching Strategy

Data	Cache	TTL / invalidation
Static assets, product images	CDN	Long TTL, versioned URLs
Sale page HTML/JSON	CDN	5–30 s; purge on SOLD_OUT
Product details	Redis cache-aside	5 min; evict on update event
Stock badge	Redis (written by Inventory)	≤ 1 s staleness acceptable
Inventory truth	Never cached for decisions	–

Part B – Reliability & Failure Handling

B1. Failure Mode Matrix

Failure	Detection	Impact	Recovery	Data loss?
Inventory pod crash mid-request	K8s liveness	Request fails	Client retries with same Idempotency-Key → replay or completes	No
Redis primary fails	Sentinel/cluster health	Gate unavailable ~5–15 s	Replica promoted; StockReconciler rebuilds stock from DB available_quantity before re-opening; during failover requests get 503 (safe)	No (DB is truth)
Redis data lost entirely	Reconciler mismatch	Gate empty	Re-seed from DB; never over-allocate because DB still enforces	No
Inventory DB primary fails	Health checks	Reservations pause ~30 s	Auto-failover to sync standby (RPO 0); app retries; waiting room holds users	No
Order DB fails	Health checks	Order writes pause	Failover; payment events wait in Kafka	No

Failure	Detection	Impact	Recovery	Data loss?
Payment gateway down / slow	Circuit breaker metrics	Can't charge	Breaker OPEN → fail fast 503; reservations held to expiry; switch to secondary provider; reconciler resolves in-flight payments	No
Payment timeout (unknown result)	Timeout	Customer unsure	TIMED_OUT → status query + webhook → resolve; never double charge	No
Payment succeeds, Order Service down 30 s	Consumer lag alert	Order confirmation delayed	Kafka redelivery + idempotent consumer + OrderReconciler	No
Kafka broker fails	Broker health	None (RF=3, min.insync=2)	Leader election	No
Outbox relay down	Unpublished-rows age alert	Events delayed	Relay restarts, resumes from DB	No
Poison message	Retry count	One message stuck	DLQ + alert + replay	No
Notification provider down	Error rate	Emails delayed	Retry, DLQ, alternate provider	No
AZ outage	Cloud health	1/3 capacity lost	Multi-AZ pods & DBs; LB routes away	No
Region outage	Global health checks	Platform down	DR region warm standby; RTO ~30–60 min, RPO seconds (async cross-region)	≤ seconds

B2. Recovery When the Database Fails (jury question)

1. Primary in AZ-a dies → managed PostgreSQL promotes **synchronous standby** in AZ-b (~30 s). Because replication is synchronous, every committed reservation/payment/order exists on the standby (**RPO = 0**).
2. During failover: Inventory returns `503 SERVICE_UNAVAILABLE` + `Retry-After`; waiting room stops admitting; clients retry with same idempotency key.
3. After failover: connection pools reconnect (PgBouncer); `StockReconciler` re-syncs Redis `stock` from DB; waiting room resumes.
4. In-flight transactions not committed are rolled back → their Redis decrements are corrected by reconciler (undersell, never oversell).

B3. Recovery When the Payment Gateway Fails (jury question)

1. Timeouts / 5xx rise → circuit breaker OPENS after 50% failures over 20 calls.
2. New payment attempts: **fail fast** (`503 PAYMENT_UNAVAILABLE`) or **route to secondary provider** via `PaymentGatewayFactory`.
3. In-flight payments (`TIMED_OUT/PENDING`): reconciler polls gateway status once it recovers; webhooks also arrive late and are processed idempotently.
4. Reservations remain held until `expires_at` (optionally **extended** by ops for this sale if the outage is gateway-side), so customers don't lose their unit due to our provider's outage.
5. `HALF_OPEN` after 30 s → 5 trial calls → CLOSED if healthy.

B4. Timeouts, Retries & Idempotency Policy (summary)

- Every remote call has a **timeout**; every retry carries the **same idempotency key**; only **idempotent operations** are retried automatically.
- Exponential backoff with jitter; max attempts bounded; then DLQ or user-visible error.
- Kafka consumers: at-least-once + `processed_event` dedupe.

B5. Stress Scenario Walkthroughs (Phase V)

#	Scenario	What happens
1	One successful purchase	Gateway auth → waiting room admits → Lua OK → DB reserve (txn) → 201 → checkout → order PAYMENT_PENDING → payment SUCCEEDED + outbox → Kafka → Inventory SOLD, Order CONFIRMED → Fulfilment PROCESSING → Notification.
2	Failed payment	Gateway declines → payment FAILED + outbox → Kafka → Inventory: guarded release (reserved-1, available+1), Redis INCRBY, SREM → <code>inventory.restocked</code> → waiting room admits next buyer → Order CANCELLED → customer notified.
3	Duplicate Buy	Same key → Redis <code>idem</code> hit → 200 same reservation. Different key, same user → <code>buyers</code> set → 409 LIMIT_REACHED. Concurrent race → DB unique constraint picks one.
4	Payment success + Order Service down 30 s	Event durable in Kafka; offset uncommitted; consumer lag alert; service recovers → redelivered → idempotent confirm. Reconciler as safety net.
5	Inventory reaches zero	Lua returns SOLD_OUT in < 1 ms; <code>inventory.sold_out</code> → sale status SOLD_OUT, CDN purge, waiting room closes.
6	50x traffic	See A2: mandatory waiting room, stock bucketing, CDN sold-out flip, tighter limits. Correctness unchanged since DB still sees only winners.
7	DB / gateway failure	See B2 / B3.

Part C – Trade-offs Accepted

Choice	Benefit	Cost
CP for inventory (consistency over availability)	Never oversell	Short reservation pauses during DB failover
AP for browse/stock badge	Always fast	Stock number may be ~1 s stale
Async post-payment	Survives downstream failures	"Confirming order..." for a few seconds; eventual consistency to explain
Pre-scaling	Ready for the spike	Pay for idle capacity ~1 hour per sale
Waiting room	Stability & fairness	Users wait in a queue

12 Security & Observability

Part A – Security

A1. Controls by Requirement

Requirement	Design
Authentication	OIDC/OAuth2 login via Identity Provider; short-lived JWT access tokens (15 min) + refresh tokens; gateway validates signature via cached JWKS (no IdP call per request). Login required before Buy Now. MFA for admin accounts.
Authorization	Role/scope based (<code>customer</code> , <code>admin</code> , <code>ops</code>); resource ownership checks in services (customer can only read own orders/reservations); service-to-service via mTLS (service mesh) + workload identity.
HTTPS / secure comms	TLS 1.2+ at CDN and LB; HSTS; mTLS inside cluster; private subnets for services & DBs; egress to gateways via NAT with allow-listed IPs.
Rate limiting & abuse	CDN/WAF bot management, IP reputation, OWASP rule set; gateway token buckets (per user, per IP, per device); waiting room with signed, single-use, short-TTL admission tokens (HMAC, bound to user ID) – cannot be shared or replayed; per-customer purchase limit (1) enforced in Redis and DB.
Input validation	Schema validation at gateway (OpenAPI), bean validation in services, quantity bounds (1..limit), UUID formats; parameterised SQL only (no injection); output encoding; size limits on bodies.
Payment data (PCI-DSS)	We never see or store card numbers : hosted checkout / gateway SDK tokenisation. We store only <code>gateway_txn_id</code> , amount, status. Webhooks verified by HMAC signature + timestamp (replay protection). Payment service in isolated namespace with restricted network policy.
Secrets management	Vault / cloud Secrets Manager; dynamic DB credentials; KMS-encrypted; no secrets in code, images or env files; automatic rotation.
Data protection	Encryption at rest (KMS) for DBs, Redis, Kafka, backups; PII (email, phone, address) minimised and masked in logs.
Audit logging	Append-only <code>audit_log</code> for every inventory/reservation/payment/order transition + admin actions (who, what, when, <code>trace_id</code>); shipped to immutable storage (WORM bucket) for 1 year.
Supply chain / infra	Signed container images, image scanning, least-privilege IAM, K8s network policies, Pod Security standards.

A2. Threats Specific to Flash Sales

Threat	Mitigation
Bots/scalpers buying all stock	WAF bot score, device fingerprinting, login + account age rules, per-customer limit, admission tokens bound to user, randomised admission
Replay / duplicate requests	Idempotency keys; signed single-use admission tokens
Price tampering	Price computed server-side by <code>PricingStrategy</code> ; client price ignored
Fake payment webhooks	HMAC signature verification + gateway status query before confirming

Threat	Mitigation
DDoS	CDN absorption, WAF, rate limits, autoscaling
Inventory manipulation by insiders	Admin actions audited, 4-eyes approval for stock changes during live sale

Part B – Observability

B1. Key Metrics (Prometheus, RED + business)

Metric	Type	Labels	Why
<code>http_requests_total</code>	counter	service, route, status	Request rate & errors
<code>http_request_duration_seconds</code>	histogram	service, route	p50/p95/p99 latency
<code>reservation_attempts_total</code>	counter	sale, result (OK/SOLD_OUT/DUPLICATE/LIMIT)	Funnel & contention
<code>reservation_failures_total</code>	counter	sale, reason (db_error/gate_error)	Inventory reservation failures
<code>inventory_available / _reserved / _sold</code>	gauge	sale	Live stock
<code>inventory_invariant_violation</code>	gauge	sale	available+reserved+sold != total OR sold > total OR Redis stock > DB available
<code>payment_attempts_total</code>	counter	provider, result	Payment failures
<code>payment_latency_seconds</code>	histogram	provider	Gateway health
<code>circuit_breaker_state</code>	gauge	dependency	OPEN/HALF_OPEN visibility
<code>orders_by_status</code>	gauge	status	Lifecycle health
<code>order_conversion_ratio</code>	gauge	sale	reservations → paid orders
<code>kafka_consumer_lag</code>	gauge	topic, group	Queue backlog
<code>outbox_unpublished_age_seconds</code>	gauge	service	Stuck events
<code>dlq_messages_total</code>	counter	topic	Poison messages
<code>reservations_expired_total</code>	counter	sale	Abandoned holds
<code>waiting_room_queue_length</code>	gauge	sale	User experience

B2. Structured Logs (JSON → Loki/ELK)

Every critical business event logs one JSON line:

```
{ "ts":"2026-10-05T10:00:01.234Z", "level":"INFO", "service":"inventory-service",
  "event":"reservation.created", "traceId":"4bf92f35...", "spanId":"00f067aa...",
  "saleId":"s-42", "reservationId":"r-77", "customerIdHash":"c9f1...", "qty":1,
  "availableAfter":37, "latencyMs":4, "idempotencyKey":"k-123" }
```

Logged events: `reservation.created/rejected/released/expired`,
`payment.initiated/succeeded/failed/timed_out/reconciled`, `order.state_changed`, `circuit.opened/closed`,
`dlq.message`, `admin.stock_changed`. PII hashed or masked; card data never logged.

B3. Distributed Tracing (OpenTelemetry → Tempo/Jaeger)

- Gateway creates/propagates W3C `traceparent`; every service instruments HTTP, gRPC, JDBC, Redis, Kafka.
- **Trace context is carried inside Kafka event envelopes** (`traceId`) so one trace spans: `POST /reservations` → `Lua` → `DB txn` → `checkout` → `payment` → `gateway` → `outbox` → `Kafka` → `order confirm` → `fulfilment` → `notification`.
- Sampling: 100% for checkout/payment/order paths; 1–5% for browse; 100% for errors (tail sampling).

B4. Dashboards

1. **Flash Sale Live** – stock gauges, reservation attempts by result, waiting-room length, sell-through curve, conversion.
2. **Payments** – success/failure/timeout rates by provider, latency, breaker state, reconciler backlog.
3. **Orders & Events** – orders by status, Kafka lag, outbox age, DLQ count.
4. **Platform** – RED metrics per service, pod counts, DB/Redis CPU & connections.

B5. Alerts (PagerDuty)

Alert	Condition	Severity
Inventory inconsistency	<code>inventory_invariant_violation > 0</code> for 1 min, or <code>sold > total</code> (ever)	P1 – page immediately
Redis vs DB stock drift	<code>Redis stock > DB available</code> for > 30 s	P2
Payment failure spike	<code>failure rate > 15%</code> over 5 min (baseline 5%)	P1
Circuit breaker open	<code>any payment breaker OPEN > 1 min</code>	P1
Queue backlog	<code>kafka_consumer_lag{group=order}</code> > 500 or growing for 2 min	P2
Stuck outbox	<code>outbox_unpublished_age_seconds > 60</code>	P2
DLQ	<code>any new DLQ message on payment/order topics</code>	P2
Paid-not-confirmed orders	<code>payments SUCCEEDED with order not CONFIRMED > 5 min</code>	P1
Latency SLO burn	<code>reserve p99 > 200 ms</code> for 5 min	P2
Error rate	<code>5xx > 2%</code> on checkout	P1

13 Architecture Decision Records

ADR-001: Microservices with a dedicated Inventory & Reservation service

Context. Flash-sale load is wildly uneven: browse and the reserve path spike 50x, while order/fulfilment volume is tiny (~100 orders). Payment must be isolated for security (PCI scope).

Options. (a) Modular monolith · (b) Microservices by business capability.

Decision. Microservices: Catalog, Sale, Cart, **Inventory & Reservation**, Checkout, Payment, Order, Fulfilment, Notification — each owning its data.

Consequences.  Scale the hot path (Inventory, Gateway) independently; failure isolation; PCI scope limited to Payment.  Distributed transactions → need Saga/outbox; operational overhead (tracing, deployment).

Mitigation: a small team could start as a modular monolith with the same boundaries.

Why this boundary? Inventory is the only place where contention on a shared resource must be serialised; giving it one owner means **one place** to reason about consistency.

ADR-002: PostgreSQL (SQL) as source of truth; Redis for speed

Context. We need *guarantees* (no oversell, no duplicate payment), not probabilities.

Option	Pros	Cons
PostgreSQL	ACID, row locks, CHECK/UNIQUE constraints, mature HA	Vertical write limits per row
NoSQL (DynamoDB/Cassandra/Mongo)	Horizontal scale	Weaker multi-item transactions; conditional writes possible (DynamoDB) but constraints across items are application-enforced
Redis only	Fastest	Not durable enough as money/stock truth

Decision. PostgreSQL for Inventory, Order, Payment; Redis for gate, cache, rate limiting, cart; OpenSearch for search.

Consequences.  Constraints make violations *impossible*.  Single-row hotspot → solved by ADR-003 gate. DB scaling via replicas/partitioning later.

ADR-003: Concurrency control = Redis Lua gate + atomic conditional UPDATE

Context. 10,000 concurrent requests for 100 units on one row.

Options. Pessimistic `SELECT FOR UPDATE` · Optimistic version check · Atomic conditional UPDATE · Redis atomic counter · Single-consumer queue per SKU · Distributed lock.

Decision. Two-layer: **Redis Lua script** (atomic check + decrement + dedupe) rejects ~99% in memory; winners execute `UPDATE ... WHERE available_quantity >= qty` with `CHECK (available_quantity >= 0)` in one ACID transaction with the reservation insert.

Consequences.  Sub-ms rejections; DB touched $\approx$ 100 times; hard guarantee in DB.  Two stores can drift $\rightarrow$ compensation + `StockReconciler`; drift only ever toward underselling. **Rejected:** optimistic locking (retry storm under contention), pessimistic alone (lock convoy), distributed locks (slow, complex). **Fallback/scale path:** queue-per-SKU or stock bucketing at 50 $\times$.

ADR-004: Temporary reservation with 10-minute TTL

Context. Units must be held while the customer pays, but abandoned holds must return to stock.

Decision. Reservation row with `expires_at`; `ReservationExpiryJob` (every 5 s, `FOR UPDATE SKIP LOCKED`) releases expired holds; guarded transitions; payment-in-flight holds are resolved via the reconciler before release.

Consequences.  No permanent loss of stock; fair re-offering.  Late-payment-after-expiry edge case $\rightarrow$ compensating refund. TTL is a business trade-off (shorter = more sales churn, longer = more blocked stock).

ADR-005: Idempotency keys on all business-creating APIs + idempotent consumers

Decision. `Idempotency-Key` header required on reserve/checkout/payment/cancel; UNIQUE DB constraints; gateway receives unique merchant reference; Kafka consumers dedupe on `event_id`.

Consequences.  Safe retries everywhere (client, gateway, Kafka).  Key storage & TTL management; clients must generate keys correctly.

ADR-006: Synchronous reservation & payment initiation; asynchronous post-payment flow

Option	Pros	Cons
Fully synchronous chain	Simple, immediate	One failure (e.g. Order down) fails/loses the purchase
Fully asynchronous (queue everything)	Most resilient	User doesn't know if they got the item; complex UX
Hybrid	Instant yes/no on stock; resilient afterwards	Eventual consistency after payment

Decision. Hybrid. Sync where the user needs a decision (reserve, checkout, payment start); async (Kafka) after payment outcome.

Consequences.  Order Service outage of 30 s loses nothing.  Order shows "confirming..." briefly; need tracing across async hops.

ADR-007: Transactional Outbox + Kafka for events

Context. Writing to DB and publishing to Kafka separately risks "paid but no event".

Options. Dual write · 2PC/XA · **Outbox** · Event sourcing.

Decision. Outbox table in each service DB, written in the same transaction; relay (Debezium CDC or poller) publishes to Kafka (RF=3, `acks=all`).

Consequences. ✅ Atomic state+event; replayable. ❌ Extra component; ~100 ms event latency; outbox cleanup job.

ADR-008: Saga with compensation instead of distributed transactions

Decision. Checkout orchestrates the synchronous steps; events choreograph the rest. Compensations: release reservation (payment failed/expired), refund (paid but can't fulfil), cancel order.

Consequences. ✅ No 2PC, services stay available. ❌ Eventual consistency; compensations must be idempotent and tested.

ADR-009: Payment resilience – circuit breaker, no blind retries, reconciliation

Decision. Resilience4j circuit breaker + 10 s timeout on charge; on timeout mark `TIMED_OUT` and resolve via status query/webhook; `PaymentReconciler` job; secondary provider through Factory/Adapter.

Consequences. ✅ No double charges, gateway outages contained. ❌ Customers may see "processing" for up to minutes in rare cases.

ADR-010: Virtual waiting room + layered rate limiting

Context. A 500k req/s spike in seconds; autoscaling too slow; bots.

Decision. CDN/WAF → gateway token buckets → waiting room issuing signed admission tokens at a controlled rate (e.g. 20k/s), closes when sold out.

Consequences. ✅ Backend sees a smooth, bounded load; fairness; bot resistance. ❌ Extra latency/UX step; another component to operate.

ADR-011: Cache strategy – cache reads, never cache inventory decisions

Options. Write-through · Write-back · **Cache-aside** · No cache.

Decision. CDN + Redis cache-aside for catalog/sale pages and the approximate stock badge (≤ 1 s stale). Inventory decisions always go through the Lua gate + DB.

Consequences. ✅ 95%+ of reads never reach services. ❌ Stock badge may briefly show "3 left" when it's 0 — reserve call returns `SOLD_OUT` correctly.

ADR-012: Kubernetes, multi-AZ, pre-scaling before sales

Decision. Stateless services on managed Kubernetes across 3 AZs; managed PostgreSQL with sync standby; Redis Cluster with replicas; Kafka 3 brokers; scheduled pre-scaling 30 min before sale; warm DR region.

Consequences. ✅ AZ failure tolerance, predictable capacity. ❌ Cost of idle pre-scaled capacity during the sale window.

Summary of Major Trade-offs

Dimension	Our choice	What we gave up
Consistency vs availability (inventory)	Consistency	Brief unavailability during DB failover
Sync simplicity vs async resilience	Hybrid	Eventual consistency after payment
Optimistic vs pessimistic	Neither alone – atomic update behind a gate	Two-store reconciliation
SQL vs NoSQL	SQL for truth , NoSQL/Redis for speed	Harder horizontal write scaling
Simplicity vs scalability	Microservices + infra components	Operational complexity

14 Pitch Plan & Jury Q&A

Pitch Timing

Time	Slide(s)	Speaker	Key line
0:00–0:30	1–2 Problem	Architect	"Read-heavy, contention-heavy, write-light: 99% must hear <i>sold out</i> fast; the 100 winners must be perfect."
0:30–1:00	3 Requirements	Architect	"Guarantees are made impossible to violate; targets are measured."
1:00–2:00	4 HLD	Architect	"Filter at the edge, serialise in exactly one place, recover asynchronously."
2:00–3:00	5–6 (7 backup) Critical design	Reliability Eng.	"Consistency is guaranteed by one SQL statement; Redis only decides who may try."
3:00–3:45	8–9 Payment & Order	Data/API Eng.	"After a timeout we never assume failure and never charge twice."
3:45–4:30	10 LLD + SOLID + Patterns	LLD Eng.	"Adding PayU is one adapter class and one factory entry."
4:30–5:00	11 Scalability & Reliability, 13 close	Architect	"Every failure errs toward underselling, never overselling."

Slide 12 is the scripted answer to the **final jury question**.

Jury Questions – Prepared Answers

Q1. Why is this the correct service boundary? Inventory is the only shared, contended resource. Giving it a single owner (Inventory & Reservation Service with its own DB) puts all contention logic in one place we can reason about and scale independently. Payment is separate for PCI isolation and external-failure containment; Order owns the lifecycle state machine. Boundaries follow data ownership and failure isolation.

Q2. Where exactly is inventory consistency guaranteed? In the Inventory DB: `UPDATE inventory SET available = available - 1 ... WHERE sale_id = ? AND available >= 1`, inside the same transaction as the reservation insert, backed by `CHECK (available_quantity >= 0)` and the conservation check `available + reserved + sold = total`.

Q3. What happens if two requests reach the inventory service at the same time? Both run the Redis Lua script; Redis executes scripts one at a time, so only one sees `stock ≥ 1`. If both somehow reached the DB, PostgreSQL row-locks the inventory row; the second UPDATE re-evaluates `available >= 1` after the first commits, matches 0 rows, and gets `409 SOLD_OUT`.

Q4. Why did you select this concurrency strategy? Optimistic locking causes a retry storm under 100:1 contention; pessimistic locking causes a lock convoy and pool exhaustion. An atomic conditional UPDATE is correct with microsecond lock hold, and the Redis gate makes sure only ~100 requests ever reach it. Fast and provably correct (ADR-003).

Q5. Why is this operation synchronous while another is asynchronous? Sync where the user needs an answer to decide (reserve: did I get it? payment start: redirect me). Async where the system needs work done reliably regardless of failures (confirm order, sell stock, ship, notify). Async with outbox + Kafka survives the Order Service outage.

Q6. How does the design recover from payment success followed by order failure? Payment + outbox commit atomically → Kafka holds the event durably → Order consumer offset not committed → redelivery after recovery → idempotent processing via `processed_event` + guarded UPDATE. Poison messages → 5 retries → DLQ + alert. `OrderReconciler` re-drives paid-but-unconfirmed orders; if impossible, compensating refund.

Q7. Which component is the likely bottleneck and how will it scale? The hot inventory key/row for the single SKU. Redis gate absorbs it (100k+ ops/s); at 50× we split stock into 10 buckets across Redis shards and optionally pre-allocate tokens to Inventory pods. The DB row only sees ~100 winners. Secondary bottleneck: payment gateway rate limits → circuit breaker + secondary provider.

Q8. What if Redis says OK but the DB transaction fails? Compensate Redis (`INCRBY` , `SREM` , delete idem key). If compensation is lost, Redis shows *fewer* units than the DB (undersell, safe); `StockReconciler` resyncs from the DB every 10 s.

Q9. What if a payment succeeds after the reservation expired? Expiry job doesn't release reservations whose payment is still unresolved. If it truly happened, Inventory tries to re-reserve; if no stock, the order is cancelled and refunded automatically (compensation) and the customer is notified.

Q10. What do you sacrifice? Availability of reservations during a DB failover (~30 s), a few seconds of "confirming order", operational complexity of microservices + Kafka, and the cost of pre-scaled capacity — in exchange for zero overselling and zero lost paid orders.

Q11. SQL or NoSQL – why? SQL for inventory/order/payment: multi-row ACID and CHECK/UNIQUE constraints make violations impossible. Redis/NoSQL for cache, gate, cart and rate limits where speed matters and data is rebuildable (ADR-002).

Q12. What happens when inventory reaches zero? Lua returns `SOLD_OUT` in < 1 ms; `inventory.sold_out` event → sale status `SOLD_OUT`, CDN purges the sale page, waiting room closes. Released units (failures/expiry) emit `inventory.restocked` and re-open admission for exactly that many buyers.

Who Owns What (everyone must be able to answer all of the above)

Member	Primary deliverables
System Architect	01 Requirements, 02 HLD (context, container, HLD, component, deployment), 08 Scalability
LLD & Design Engineer	03 Class, sequences, states; 06 SOLID; 07 Patterns
Data & API Engineer	04 ER + DB design; 05 API spec + events
Reliability Engineer	03 Concurrency + Payment/Order design; 08 Reliability; 09 Security & Observability; 10 ADRs

15 Practical Scenario Walkthroughs

Each trace below follows one request through the system using the components defined in the earlier chapters.

15.1 One successful purchase (request arrival → order confirmed)

1. Customer sends `POST /v1/sales/{saleId}/reservations` with a JWT and `Idempotency-Key: k-123`.
2. CDN/WAF checks the bot score and forwards; the API Gateway verifies the JWT and the per-user rate limit.
3. The waiting room admits the customer and issues a signed admission token (TTL 60 s).
4. Inventory Service checks `idem:k-123` in Redis (not seen), then runs `reserve.lua`: stock 37 → 36, customer added to the buyers set, returns OK.
5. One PostgreSQL transaction: conditional `UPDATE inventory` (1 row), `INSERT inventory_reservation` (status RESERVED, expires in 10 min), `INSERT outbox_event`, `COMMIT`. Response **201** with `reservationId`.
6. Customer calls `POST /v1/checkout`: Checkout validates the reservation, prices it with the `FlashSalePricing` strategy, creates order O-55 (`PAYMENT_PENDING`) and marks the reservation `PAYMENT_PENDING`.
7. Payment Service inserts payment P-1 (unique idempotency key and merchant reference), charges the gateway through the circuit breaker, receives *captured*.
8. One transaction: payment → `SUCCEEDED` + outbox `payment.succeeded`. The relay publishes it to Kafka.
9. Inventory consumes it: reservation → `SOLD`, reserved-1, sold+1. Order consumes it: O-55 → `CONFIRMED`, outbox `order.confirmed`.
10. Fulfilment books the courier (`PROCESSING`); Notification sends email and SMS.

15.2 One failed payment and reservation release

1. Steps 1–7 as above, but the gateway declines the card.
2. Payment → `FAILED` + outbox `payment.failed`; customer gets **402**.
3. Inventory consumes the event: `UPDATE reservation SET status='RELEASED' WHERE id=? AND status IN ('RESERVED', 'PAYMENT_PENDING')` → 1 row → available+1, reserved-1; Redis `INCRBY stock 1`, `SREM buyers`.
4. `inventory.restocked` → the waiting room admits one more buyer. Order O-55 → `CANCELLED`; customer notified.
5. A duplicate `payment.failed` delivery matches 0 rows in step 3 and does nothing — no double release.

15.3 Duplicate "Buy" request

- **Same key (double click / client retry):** Redis `idem:k-123` hit → **200** with the same reservation R-77. If Redis missed it, the DB `UNIQUE(idempotency_key)` rejects the insert and the existing reservation is returned.
- **Same customer, new key (two tabs):** Redis `buyers` set → **409 LIMIT_REACHED**; DB partial unique index (`sale_id`, `customer_id`) backs it up.
- **Two identical requests at the same instant:** Lua scripts run one at a time, and the DB unique constraint is the final arbiter — exactly one reservation.

15.4 Payment succeeds, then the Order Service fails for 30 s

1. Payment → `SUCCEEDED` and outbox row are committed atomically. The customer sees "Payment received, confirming order...".
2. The relay publishes `payment.succeeded` to Kafka (RF=3, acks=all). Inventory consumes it normally (unit SOLD).
3. Order pods are down: the consumer offset is not committed, so the event stays in Kafka. A consumer-lag alert fires.
4. After ~30 s Kubernetes restores the pods; Kafka redelivers the event. Order inserts `processed_event(E-1)` (dedupe), runs a guarded `UPDATE ... WHERE status='PAYMENT_PENDING' → CONFIRMED`, commits, then commits the offset.
5. If processing kept failing: 5 retries with backoff → DLQ + alert → replay after the fix. The `OrderReconciler` (every minute) re-drives any order that has a `SUCCEEDED` payment but is not confirmed after 2 minutes; if it can never be confirmed, it triggers a **compensating refund**.

15.5 Inventory reaches zero

The Lua gate returns `SOLD_OUT` in under 1 ms to every further request. Inventory publishes `inventory.sold_out`: the Sale Service sets the sale to `SOLD_OUT`, the CDN page is purged to show "Sold out", and the waiting room closes its queue. If units come back (payment failure or expiry), `inventory.restocked` re-opens admission for exactly that many buyers.

15.6 Traffic increases 50×

The waiting room becomes mandatory with randomised fair admission; stock is split into 10 Redis keys on different shards (stock bucketing) so no single key is hot; the CDN flips to a static sold-out page instantly; gateway, waiting room and inventory pods are pre-scaled 30 minutes before the sale; rate limits tighten per IP and device. Correctness does not change because the database still only sees ~100 winners.

15.7 Database or payment gateway failure

- **Inventory DB primary fails:** the synchronous standby is promoted in ~30 s with RPO = 0. Reservations return **503 + Retry-After** meanwhile; clients retry with the same idempotency key. Afterwards the `StockReconciler` resyncs Redis from the DB.
- **Payment gateway fails:** the circuit breaker opens after 50% failures over 20 calls; new attempts fail fast (**503 PAYMENT_UNAVAILABLE**) or are routed to a secondary provider; in-flight payments are resolved by the reconciler via status queries and late webhooks; reservations stay held until expiry.

16 Appendix: Deliverables Index

#	Deliverable	In this document	Submission file
1	Requirements & Assumptions	Ch. 2	01_Requirements/Requirements_and_Assumptions.md
2	System Context Diagram	Fig. 3.1	02_HLD/System_Context.png
3	HLD Architecture	Ch. 3, Fig. 3.2	02_HLD/HLD_Architecture.png, HLD_Explanation.md
4	Container Diagram	Fig. 4.1	02_HLD/Container_Diagram.png
5	Component Diagram	Fig. 4.2	02_HLD/Component_Diagram.png
6	Deployment Diagram	Fig. 4.3	02_HLD/Deployment_Diagram.png
7	Database / ER Diagram	Ch. 9	04_Database/ER_Diagram.png, Database_Design.md
8	Class Diagram	Ch. 7	03_LLD/Class_Diagram.png
9	Purchase / Reservation Sequence	Ch. 5	03_LLD/Sequence_Purchase_Reservation.png
10	Payment Sequence	Ch. 6	03_LLD/Sequence_Payment.png
11	Order Sequence	Ch. 6	03_LLD/Sequence_Order.png
12	Order / Reservation State Diagram	Ch. 5, 6	03_LLD/State_Reservation.png, State_Order.png
13	SOLID Mapping	Ch. 8	06_SOLID/SOLID_Mapping.md
14	Design Pattern Mapping	Ch. 8	07_Design_Patterns/Design_Pattern_Mapping.md
15	API Specification	Ch. 10	05_API/API_Specification.md, openapi.yaml
16	Scalability & Reliability Design	Ch. 11	08_Scalability_Reliability/Scalability_and_Reliability.md
17	Security & Observability Design	Ch. 12	09_Security_Observability/Security_and_Observability.md
18	Architecture Decision Record	Ch. 13	10_ADR/Architecture_Decision_Records.md
19	Final Presentation	—	12_Presentation/SALESTORM_Pitch.pptx